

OMR Corner Detection

Technical review: library, algorithm, thresholds, and the accept/reject gate behind the live "X/4 corner marks" indicator and the post-capture scan validation.

1. Library / Dependency

opencv_dart ^2.2.1 (resolved 2.2.1+4)  Dart FFI bindings to real OpenCV, backed by the dartcv4 native package (see pubspec.yaml). This is general-purpose OpenCV, not a document-scanner or fiducial-marker-specific library, and specifically not ArUco or AprilTag: there is no marker dictionary, no per-marker identity, and no built-in error correction. The "fiducial marks" referenced throughout the code are plain printed black squares (see Appendix D, tool/generate_sheets.dart, drawRect) located purely by generic blob detection, not a fiducial-recognition algorithm.

2-4. Functions, Call Sites, and Detection Method

All detection logic lives in lib/core/omr/omr_decoder.dart. There are two independent detection stages, run in sequence for every corner search.

Stage A find the page boundary (`_detectPageQuad`, line 461)

gaussianBlur -> canny (edges) -> getStructuringElement + dilate (bridge gaps) -> findContours -> for the largest sufficiently-big, sufficiently-central contour: arcLength + approxPolyDP to fit a 4-point convex polygon; if that fails, minAreaRect + .points as a fallback rotated rectangle. Method: Canny edge detection + contour extraction + polygon approximation, with a bounding-rectangle fallback. No Hough transform is used anywhere in this file.

Stage B find each of the 4 printed corner squares (`_findMarkerNear`, line 604)

Run once per corner, near wherever Stage A predicted it should be (`_seedFor`, line 349): createCLAHE + apply (local contrast boost) -> gaussianBlur -> adaptiveThreshold -> dilate -> findContours -> filter candidate blobs by size / fill-ratio / aspect / contrast -> pick the survivor closest to the expected position. Method: adaptive thresholding + contour extraction + shape/contrast filtering. This is blob detection of a known printed shape, not a classical corner-point detector (no Harris, no goodFeaturesToTrack) despite the "corner" naming.

Call sites

Caller	Function	Runs on
exam_scanning_screen.dart :20	locateCorners -> _refineCorners	full-res captured photo  post-capture gate
exam_scanning_screen.dart :33	checkCornersFromLuma	live low-res preview frame  on-screen guide
app_state.dart:23	decode -> _refineCorners	full-res captured photo  during "Compile Data"

5. Thresholds / Parameters

All defined in omr_decoder.dart. Only the Canny thresholds are computed from the photo itself; every other value below is a fixed constant.

Constant	Value	Line	Controls
_cornerSearchWindowFrac	0.08	28	search radius around a good seed

<code>_cornerSearchWindowFrac</code> <code>NoPageSeed</code>	0.22	39	search radius when Stage A found nothing
Canny thresholds	mean x (0.67, 1.33)	477-487	the one adaptive parameter derived per-photo
<code>_markerMinContrast</code>	20 (gray levels)	77	how much darker than local background a blob must be
<code>_markerMinFillRatio</code>	0.55	86	how solidly a blob must fill its own bounding box
<code>_markerMaxAspect</code>	7.0	94	max elongation before a blob is rejected as a line/text
aspect-ratio tolerance	+/-25% of expected	434	<code>_validateQuad</code> 's sheet-shape check
min quad area	5% of image area	442	<code>_validateQuad</code> 's too-small/too-far check
<code>_blankFillFloor</code>	0.28	68	bubble-fill floor for "marked" (bubble scoring, not corner detection)
<code>_ambiguousMargin</code>	0.15	73	same, bubble scoring

6. How 0/4 ... 4/4 Is Computed

`checkCornersFromLuma` (line 164) runs `_findMarkerNear` independently for each of the 4 template corners and returns a `List<bool>` where there is no shape/rectangle check between them; each corner stands alone. `exam_scanning_screen.dart` counts `.where((f) => f).length` for the on-screen "X/4" text and drives the per-corner bracket colors from that same list. This computation runs on the live camera preview frame (`plane.bytes` from `CameraImage`), not on the eventual captured photo.

7. Validation Before a Scan Proceeds

Independent of the live 0-4/4 indicator: on the actual captured photo, `_refineCorners` (line 371) requires literally all 4 `_findMarkerNear` calls to succeed where no reconstructing a missing one from the other 3, and no "use anyway" override. Then `_validateQuad` (line 412) checks the 4 found points are correctly ordered, within 25% of the expected aspect ratio, and cover at least 5% of the frame. Any failure throws a `StateError`, the photo is discarded, and `decode()` where the perspective warp + bubble sampling is never reached.

Weaknesses, Plainly

Assessed directly against the implementation above, not against a wishlist.

1. **The two stages are coupled through the search window, silently.** Stage B only looks within `_cornerSearchWindowFrac / ...NoPageSeed` of wherever Stage A guessed. If Stage A's guess is off by more than that window it's plausible on a warped sheet, unusual lens distortion, or a badly broken contour the true mark is never even examined, and it simply reads as "not found." There is no signal distinguishing "no mark was there" from "we looked in the wrong place."
2. **Almost every threshold is a hand-picked constant, not a fitted one.** The file's own comments admit this directly: `_blankFillFloor`'s comment says it was tuned only against blank sheets and "has never been tested against a sheet with genuine pencil marks"; `_validateQuad`'s aspect-tolerance comment calls 0.25 "a middle ground" reached by tightening then loosening once each. None of these came from a systematic dataset.
3. **Flatness between the 4 corners is never checked.** `_validateQuad` verifies the 4 corner points form a plausible rectangle it has no way to know whether the paper stayed flat in between them. A prior test batch produced a sheet with 72/72 items flagged despite passing this gate: all 4 corners can be correct and the interior can still be wrong if the paper itself was not flat when photographed.
4. **No real fiducial-marker properties.** A true ArUco/AprilTag marker self-identifies (so top-left can never be confused with top-right) and carries error-correction bits. These are unmarked black squares distinguished purely by expected position if two candidate blobs in a search window are both plausible, the position-vs-contrast scoring (score = distance / contrast, line 699) is the only tiebreaker, and it can be fooled by a closer, high-contrast piece of clutter beating a farther, fainter real mark.

This document is a snapshot of the implementation as of the review date below. The full source of the files it discusses is reproduced in the appendices that follow, so any claim above can be checked directly against the code rather than against this summary.

```

1 import 'dart:io';
2 import 'dart:math' as math;
3 import 'dart:typed_data';
4
5 import 'package:opencv_dart/opencv_dart.dart' as cv;
6
7 import '../models/omr_scan_result.dart';
8 import 'omr_templates.dart';
9
10 /// Result of [OmrDecoder.locateCorners]  whether all 4 fiducial marks were
11 /// found for a given photo + template, without doing the (much more
12 /// expensive) perspective warp and bubble sampling.
13 class AlignmentCheck {
14   final bool aligned;
15   final String? message;
16   const AlignmentCheck.aligned() : aligned = true, message = null;
17   const AlignmentCheck.misaligned(this.message) : aligned = false;
18 }
19
20 /// Decodes a photographed answer sheet into marked choices, using the
21 /// fiducial corner markers in [OmrExamTemplate.cornerMarkers] to correct for
22 /// skew/perspective before sampling each [BubblePos].
23 class OmrDecoder {
24   const OmrDecoder();
25
26   /// How far (as a fraction of the shorter image dimension) to search around
27   /// each corner marker's expected position for the actual fiducial mark.
28   static const double _cornerSearchWindowFrac = 0.08;
29
30   /// Wider fallback search radius used when [_detectPageQuad] found no page
31   /// boundary at all (e.g. the sheet's white paper against a similarly
32   /// light background  common with a white desk/table  gives Canny
33   /// nothing to find an edge on). Without a detected boundary, the seed
34   /// position for each corner falls back to a naive "page fills the whole
35   /// frame" guess, which is frequently wrong by more than the normal
36   /// window can absorb once any background is visible around the sheet.
37   /// Trades a bit of extra search cost for not failing outright whenever
38   /// the background happens to be low-contrast against the paper.
39   static const double _cornerSearchWindowFracNoPageSeed = 0.22;
40
41   /// Canonical pixels per PDF point when warping the sheet flat. Bubble
42   /// sampling geometry below is tuned against this scale.
43   static const double _canonicalPxPerPt = 2.0;
44
45   /// Block size (must be odd) for the adaptive threshold that binarizes the
46   /// warped sheet before bubble sampling. Large enough to span several
47   /// bubbles so it tracks slow lighting gradients across the page rather
48   /// than reacting to a single bubble, small enough to still adapt to
49   /// vignetting/uneven light within the sheet.
50   static const int _adaptiveThresholdBlockSize = 45;
51
52   /// Constant subtracted from the local adaptive-threshold mean; higher
53   /// values require darker pixels to count as "ink".
54   static const double _adaptiveThresholdC = 12;
55
56   /// A bubble's ink fill fraction (0 = empty, 1 = fully black) must clear
57   /// this floor to be considered marked at all. Raised from 0.15 (first
58   /// correction) to 0.22 (halved the flagged rate on a blank OLSAT sheet,
59   /// ~50-58% -> ~19-32%), now to 0.28: geometry was confirmed pixel-perfect
60   /// (see decode()/saveDebugVisualization commit history), so the remaining
61   /// flags on a blank sheet are purely fill-noise, not misaligned sampling
62   ///  the right lever is still this floor. Re-check against a blank-sheet
63   /// scan after this change; if the flagged rate keeps dropping
64   /// proportionally, keep raising it, but this has never been tested
65   /// against a sheet with genuine pencil marks  at some point a real,
66   /// light mark could start reading as blank, so this can't go up forever
67   /// without that check too.
68   static const double _blankFillFloor = 0.28;
69
70   /// The most-filled bubble in an item must beat the runner-up fill
71   /// fraction by at least this much to be treated as an unambiguous single
72   /// mark.
73   static const double _ambiguousMargin = 0.15;
74
75   /// A fiducial blob must be at least this many gray levels darker than the
76   /// local background to count as a real mark (not a shadow edge).
77   static const double _markerMinContrast = 20;
78
79   /// A candidate blob's contour area must fill at least this fraction of
80   /// its own bounding box to count as a mark. The printed marks (solid
81   /// square or thin tick  see [_findMarkerNear]) are both clean filled
82   /// shapes; background clutter that happens to be dark and roughly the
83   /// right size (fabric texture, shadows, a keyboard key) rarely fills its
84   /// bounding box this tightly, so this rejects that clutter instead of
85   /// confidently reporting a mark "found" on whatever's actually in frame.
86   static const double _markerMinFillRatio = 0.55;
87
88   /// A candidate blob's bounding box must not be more elongated than this
89   /// (longer side / shorter side). The thinnest legitimate mark is the
90   /// tick (~2.5pt stroke over a 12pt span, aspect ~4.8), so this allows
91   /// real photo blur/perspective slack above that while still rejecting

```

```

92  /// long edges █ a table line, a run of header text █ that happen to
93  /// cross the search window.
94  static const double _markerMaxAspect = 7.0;
95
96  /// Inner sample radius as a fraction of the outer bubble sample radius.
97  /// The printed choice letter sits in this center zone; subtracting its
98  /// ink contribution from the outer reading isolates pencil marks in the
99  /// bubble ring. Sized against the sheet generator's current geometry
100  /// (kBubbleRadius: 6, kLetterFontSize: 6 in tool/generate_sheets.dart,
101  /// bold): a wide glyph like "W" or "M" has an advance width close to
102  /// 0.85x its font size, so half-width █ 0.42x the bubble radius █ 0.4
103  /// left letters like that poking just outside the exclusion zone,
104  /// reading as ink on an otherwise-blank bubble. 0.6 covers every glyph
105  /// with margin while still leaving most of the ring (64% of the sample
106  /// area) free to detect an actual pencil mark.
107  static const double _bubbleInnerSampleFrac = 0.6;
108
109  /// Cheap alignment check for right after a photo is captured: does the
110  /// same image load + corner search as [decode], but skips the perspective
111  /// warp and bubble sampling, so it's reasonable to run on every capture
112  /// rather than only at full-decode time.
113  AlignmentCheck locateCorners(String imagePath, OmrExamTemplate template) {
114    final src = cv.imread(imagePath);
115    try {
116      if (src.isEmpty) {
117        return const AlignmentCheck.misaligned('Could not read the captured photo.');[locateCorners], but against a live camera
131  /// preview frame instead of a captured file █ cheap enough to run
132  /// periodically while framing, for the on-screen guide's live
133  /// per-corner feedback. [lumaBytes] is the Y (luma) plane of a YUV
134  /// camera frame, already 8-bit grayscale with no color conversion
135  /// needed. [bytesPerRow] may exceed [width] (row padding, common on
136  /// Android camera buffers) █ the extra columns are cropped off.
137  ///
138  /// Returns whether each of the 4 fiducial marks is currently found,
139  /// independently, in [OmrExamTemplate.cornerMarkers] order (top-left,
140  /// top-right, bottom-left, bottom-right) █ unlike [locateCorners], this
141  /// doesn't require all 4 to form a plausible rectangle together, since
142  /// each corner's little viewfinder square needs to react on its own as
143  /// the user moves the sheet, not just an aggregate pass/fail.
144  ///
145  /// Seeds each corner's search the same way [_refineCorners] does (see
146  /// [_seedFor]) █ including its "page fills the frame" fallback when
147  /// [_detectPageQuad] finds no boundary. An earlier version of this method
148  /// skipped that fallback and reported all 4 marks "not found" outright
149  /// whenever no page-shaped boundary was visible, on the theory that a
150  /// false "aligned" (inviting a bad capture) was worse than a false "not
151  /// yet". In practice that made the live guide permanently red for the
152  /// single most common good framing █ the sheet filling most or all of
153  /// the frame, exactly as the on-screen guide asks for █ since a
154  /// full-frame sheet has no surrounding background left to detect a page
155  /// *edge* against, by definition, even though the corner marks
156  /// themselves are perfectly visible. The real defense against a false
157  /// "aligned" on random clutter is (and always was) [_findMarkerNear]'s
158  /// own contrast/fill-ratio/aspect filtering, not this page-boundary
159  /// gate █ and the post-capture [locateCorners]/[decode] path already
160  /// relies on exactly that same fallback+filtering combination as its
161  /// real accept/reject gate, so this brings the live preview in line with
162  /// logic already trusted there instead of being stricter for no proven
163  /// benefit.
164  List<bool> checkCornersFromLuma(
165    Uint8List lumaBytes,
166    int width,
167    int height,
168    int bytesPerRow,
169    OmrExamTemplate template,
170  ) {
171    final full = cv.Mat.fromList(height, bytesPerRow, cv.MatType.CV_8UC1, lumaBytes);
172    try {
173      final gray = bytesPerRow == width ? full : full.region(cv.Rect(0, 0, width, height));
174      try {
175        final pageQuad = _detectPageQuad(gray);
176        return List.generate(template.cornerMarkers.length, (i) {
177          final (expectedX, expectedY, windowFrac) = _seedFor(template.cornerMarkers[i], pageQuad, gray.width,
178            gray.height);
179          return _findMarkerNear(gray, expectedX, expectedY, windowFrac: windowFrac) != null;
180        });
181      } finally {
182      }
183    }
184  }

```

```

181     if (!identical(gray, full)) gray.dispose();
182   }
183   } finally {
184     full.dispose();
185   }
186 }
187
188 /// Shared by [locateCorners] and [checkCornersFromLuma] so a captured
189 /// photo and a live preview frame are judged by identical logic.
190 AlignmentCheck _checkAlignment(cv.Mat gray, OmrExamTemplate template) {
191   try {
192     _refineCorners(gray, template);
193     return const AlignmentCheck.aligned();
194   } on StateError catch (e) {
195     return AlignmentCheck.misaligned(e.message);
196   }
197 }
198
199 /// Reads [imagePath] against [template] and returns the decoded marks.
200 /// Throws a [StateError] with a user-facing message if the sheet couldn't
201 /// be read or aligned.
202 OmrScanResult decode(String imagePath, OmrExamTemplate template) {
203   final src = cv.imread(imagePath);
204   try {
205     if (src.isEmpty) {
206       throw StateError('Could not read the captured photo at $imagePath.');
```

```

272 /// are, instead of guessing from decoded results alone. [outputDir] must
273 /// already exist and be writable.
274 void saveDebugVisualization(String imagePath, OmrExamTemplate template, String outputDir, int pageIndex) {
275   final src = cv.imread(imagePath);
276   try {
277     if (src.isEmpty) return;
278     final gray = cv.cvtColor(src, cv.COLOR_BGR2GRAY);
279     try {
280       List<cv.Point2f> corners;
281       try {
282         corners = _refineCorners(gray, template);
283       } on StateError catch (e) {
284         // Corner detection itself failed  still write the raw photo so
285         // framing/lighting/orientation can be inspected, plus the exact
286         // error, instead of silently producing no debug output for
287         // precisely the failing case that needs to be seen.
288         cv.imwrite('$outputDir/sheet${pageIndex}_FAILED.jpg', src);
289         File('$outputDir/sheet${pageIndex}_error.txt').writeAsStringSync(e.message);
290         return;
291       }
292
293       final cornersDebug = src.clone();
294       try {
295         for (final c in corners) {
296           cv.circle(cornersDebug, cv.Point(c.x.round(), c.y.round()), 16, cv.Scalar(0, 0, 255), thickness: 5);
297         }
298         cv.imwrite('$outputDir/sheet${pageIndex}_corners.jpg', cornersDebug);
299       } finally {
300         cornersDebug.dispose();
301       }
302
303       final canonicalWidth = (template.pageWidthPt * _canonicalPxPerPt).round();
304       final canonicalHeight = (template.pageHeightPt * _canonicalPxPerPt).round();
305       final dstCorners = cv.VecPoint2f.fromList([
306         for (final corner in template.cornerMarkers)
307           cv.Point2f(corner.xFrac * canonicalWidth, corner.yFrac * canonicalHeight),
308       ]);
309       final srcCorners = cv.VecPoint2f.fromList(corners);
310       final transform = cv.getPerspectiveTransform2f(srcCorners, dstCorners);
311       try {
312         final warped = cv.warpPerspective(src, transform, (canonicalWidth, canonicalHeight));
313         try {
314           for (final section in template.sections) {
315             for (final entry in section.items.entries) {
316               for (final bubble in entry.value) {
317                 final x = (bubble.xFrac * canonicalWidth).round();
318                 final y = (bubble.yFrac * canonicalHeight).round();
319                 cv.circle(warped, cv.Point(x, y), 3, cv.Scalar(0, 0, 255), thickness: -1);
320               }
321             }
322           }
323           cv.imwrite('$outputDir/sheet${pageIndex}_grid.jpg', warped);
324         } finally {
325           warped.dispose();
326         }
327       } finally {
328         transform.dispose();
329         srcCorners.dispose();
330         dstCorners.dispose();
331       }
332     } finally {
333       gray.dispose();
334     }
335   } finally {
336     src.dispose();
337   }
338 }
339
340 /// Seeds an expected pixel position + search-window radius for one
341 /// corner marker, shared by [_refineCorners] and [checkCornersFromLuma]
342 /// so a captured photo and a live preview frame search for each mark
343 /// starting from identical assumptions. Bilinearly interpolates within
344 /// [pageQuad] when [_detectPageQuad] found a page boundary; otherwise
345 /// falls back to assuming the page fills the frame edge-to-edge
346 /// (correct when zoomed in tight enough that no page edge is visible 
347 /// see [_detectPageQuad]'s doc comment), searching a wider radius since
348 /// that fallback seed is a much rougher guess.
349 (double, double, double) _seedFor(OmrCorner corner, List<(double, double)>? pageQuad, int width, int height) {
350   if (pageQuad != null) {
351     final seed = _bilinearInterpolate(pageQuad, corner.xFrac, corner.yFrac);
352     return (seed.$1, seed.$2, _cornerSearchWindowFrac);
353   }
354   return (corner.xFrac * width, corner.yFrac * height, _cornerSearchWindowFracNoPageSeed);
355 }
356
357 /// The template's 4 [OmrCorner]s are ordered top-left, top-right,
358 /// bottom-left, bottom-right. For each, seed an expected pixel position
359 /// and search a local window around it for the actual fiducial mark (a
360 /// small dark blob, shape-agnostic beyond the fill-ratio/aspect checks
361 /// below), returning its refined centroid.
362 ///

```

```

363 /// The seed comes from [_detectPageQuad] when the photo shows background
364 /// around the sheet (the common case  the viewfinder doesn't crop tight
365 /// to the page), bilinearly interpolating within the detected page
366 /// boundary. Assuming the fractional page position maps directly onto the
367 /// full image (xFrac * image.width) is only correct when the page fills
368 /// the frame edge-to-edge, which produced systematically wrong seeds -
369 /// and therefore a skewed homography and misread bubbles - whenever any
370 /// background was visible around the sheet.
371 List<cv.Point2f> _refineCorners(cv.Mat gray, OmrExamTemplate template) {
372   const labels = ['top-left', 'top-right', 'bottom-left', 'bottom-right'];
373   final corners = template.cornerMarkers;
374   final pageQuad = _detectPageQuad(gray);
375   final found = List<cv.Point2f>().generate(corners.length, (i) {
376     final (expectedX, expectedY, windowFrac) = _seedFor(corners[i], pageQuad, gray.width, gray.height);
377     return _findMarkerNear(gray, expectedX, expectedY, windowFrac: windowFrac);
378   });
379
380   // All 4 fiducial marks must be found directly  no reconstructing a
381   // missing one from the other 3. An earlier version tolerated exactly 1
382   // missing corner, estimating it via parallelogram-diagonal math from
383   // the other 3 (a photographed rectangle's diagonals share a midpoint
384   // under mild perspective distortion). That let a scan proceed on an
385   // estimate rather than a confirmed detection  precisely for the
386   // occluded/uncertain cases (a thumb, a crease, a glare spot) most
387   // likely to also be skewed, which is the opposite of where an estimate
388   // should be trusted. Requiring a clean 4/4 makes "not enough was
389   // detected" a hard rejection instead of a silent guess feeding into
390   // perspective correction and bubble sampling.
391   final missing = [for (var i = 0; i < found.length; i++) if (found[i] == null) i];
392   if (missing.isNotEmpty) {
393     final names = missing.map((i) => labels[i]).join(', ');
394     throw StateError(
395       'Could not find the $names alignment mark${missing.length > 1 ? 's' : ''}. '
396       'Page not fully detected. Please align the sheet and try again.',
397     );
398   }
399
400   final resolved = found.cast<cv.Point2f>();
401   _validateQuad(resolved, template, gray);
402   return resolved;
403 }
404
405 /// Finding a* plausible small dark blob near each of the 4 expected
406 /// corner spots isn't enough on its own  if the sheet is out of frame or
407 /// badly misaligned, those 4 windows can each still latch onto unrelated
408 /// clutter (shadows, texture, edges) that individually passes the
409 /// size filter, so the corner search silently "succeeds" on nonsense.
410 /// This checks the 4 found points actually form a plausible sheet-shaped
411 /// rectangle together before accepting them.
412 void _validateQuad(List<cv.Point2f> corners, OmrExamTemplate template, cv.Mat gray) {
413   final tl = corners[0], tr = corners[1], bl = corners[2], br = corners[3];
414
415   if (tl.x >= tr.x || bl.x >= br.x || tl.y >= bl.y || tr.y >= br.y) {
416     throw StateError(
417       'The detected corner marks are not arranged like the sheet. Retake the photo with the sheet upright and all
418       four corners visible.',
419     );
420   }
421
422   final width = ((tr.x - tl.x) + (br.x - bl.x)) / 2;
423   final height = ((bl.y - tl.y) + (br.y - tr.y)) / 2;
424   final expectedAspect = template.pageWidthPt / template.pageHeightPt;
425   final actualAspect = width / height;
426   // A real sheet's aspect ratio should match reasonably closely (paper
427   // dimensions are exact), but this is a coarse secondary check  the
428   // centrality filter in _detectPageQuad is the primary defense against
429   // picking up background clutter (a second sheet, wall, furniture)
430   // instead of the actual sheet. 0.15 (tightened from an original 0.35
431   // that let a clutter-quad through) turned out to reject too many
432   // genuine handheld photos too, whose measured aspect ratio shifts more
433   // than that from normal perspective/keystone distortion at a
434   // non-perfectly-perpendicular angle. 0.25 is a middle ground.
435   if ((actualAspect - expectedAspect).abs() > expectedAspect * 0.25) {
436     throw StateError(
437       'The detected corners don't form a sheet-shaped rectangle. Retake the photo with just the one sheet, on a
438       plain background, in frame.',
439     );
440   }
441
442   final imageArea = gray.width * gray.height;
443   final quadArea = width * height;
444   if (quadArea < imageArea * 0.05) {
445     throw StateError('The sheet appears too small in this photo. Move closer and retake.');
```

```

451 /// page already fills the whole frame, so there's no visible boundary
452 /// edge to detect - callers should fall back to treating the image
453 /// itself as the page).
454 ///
455 /// Requiring the candidate's centroid to be reasonably central rules out
456 /// unrelated background objects - a second sheet lower in frame, a wall
457 /// corner, furniture - that can otherwise out-area the actual sheet and
458 /// get picked as "the page" just because they're bigger, even though the
459 /// on-screen guide already tells the user to center the sheet they're
460 /// scanning.
461 List<(double, double)>? _detectPageQuad(cv.Mat gray) {
462   final blurred = cv.gaussianBlur(gray, (5, 5), 0);
463   try {
464     // Canny thresholds derived from this photo's own mean brightness
465     // (the well-known "auto Canny" approach, substituting mean for the
466     // more common median here since a mean is already computed elsewhere
467     // in this file with the same `.mean()` call and a photographed sheet's
468     // grayscale histogram is roughly unimodal, so the two track closely)
469     // rather than a fixed 50/150 tuned against whatever lighting one past
470     // test photo happened to have. Classroom photos vary a lot in
471     // exposure  a fixed pair of absolute thresholds is either too
472     // sensitive (picks up desk texture/shadow as edges) on a bright photo
473     // or misses the actual page boundary on a dim one; scaling both
474     // thresholds off the photo's own brightness keeps the same relative
475     // sensitivity across that range instead of guessing one setting for
476     // an unknown "typical" photo.
477     final meanScalar = blurred.mean();
478     double meanIntensity;
479     try {
480       meanIntensity = meanScalar.val1;
481     } finally {
482       meanScalar.dispose();
483     }
484     const cannySigma = 0.33;
485     final lower = (meanIntensity * (1 - cannySigma)).clamp(0, 255);
486     final upper = (meanIntensity * (1 + cannySigma)).clamp(0, 255);
487     final edges = cv.canny(blurred, lower.toDouble(), upper.toDouble());
488     try {
489       final kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5));
490       try {
491         final dilated = cv.dilate(edges, kernel, iterations: 2);
492         try {
493           final (contours, hierarchy) = cv.findContours(dilated, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE);
494           try {
495             final imageArea = (gray.width * gray.height).toDouble();
496             final centerX = gray.width / 2;
497             final centerY = gray.height / 2;
498             final maxCenterDistance = 0.3 * math.sqrt(gray.width * gray.width + gray.height * gray.height) / 2;
499             List<(double, double)>? bestQuad;
500             double bestArea = 0;
501             for (final contour in contours) {
502               final area = cv.contourArea(contour);
503               if (area < imageArea * 0.2 || area <= bestArea) continue;
504
505               List<(double, double)>? points;
506               final peri = cv.arcLength(contour, true);
507               final approx = cv.approxPolyDP(contour, 0.02 * peri, true);
508               try {
509                 if (approx.length == 4 && cv.isContourConvex(approx)) {
510                   points = [for (final p in approx) (p.x.toDouble(), p.y.toDouble())];
511                 }
512               } finally {
513                 approx.dispose();
514               }
515
516               // approxPolyDP didn't resolve a clean convex quad  a
517               // shadow, a crease, or clutter touching the page's edge
518               // commonly breaks the boundary into >4 vertices even though
519               // it's still roughly rectangular. Falling back to the
520               // contour's minimum-area bounding rectangle instead of
521               // discarding it outright means a page whose edge is
522               // well-formed apart from one broken segment still seeds a
523               // real corner search, rather than falling all the way back
524               // to the much coarser "page fills the frame" guess
525               // (_cornerSearchWindowFracNoPageSeed) every time polygon
526               // approximation isn't perfectly clean.
527               if (points == null) {
528                 final rect = cv.minAreaRect(contour);
529                 try {
530                   // Guard against accepting a non-rectangular blob (an L
531                   // shape, a diagonal object) just because it's large and
532                   // central: a genuine page edge should fill most of its
533                   // own minimum-area rectangle, so require the contour to
534                   // cover a healthy majority of that box's area before
535                   // trusting the box as a page-quad substitute.
536                   final boxArea = rect.size.width * rect.size.height;
537                   if (boxArea <= 0 || area < boxArea * 0.6) continue;
538                   final boxPts = rect.points;
539                   try {
540                     points = [for (final p in boxPts) (p.x, p.y)];
541                   } finally {

```

```

542         boxPts.dispose();
543     }
544     } finally {
545         rect.dispose();
546     }
547 }
548
549 final centroidX = points.map((p) => p.$1).reduce((a, b) => a + b) / 4;
550 final centroidY = points.map((p) => p.$2).reduce((a, b) => a + b) / 4;
551 final centerDistance = math.sqrt(math.pow(centroidX - centerX, 2) + math.pow(centroidY - centerY,
2));
552
553 if (centerDistance > maxCenterDistance) continue;
554 bestArea = area;
555 bestQuad = points;
556 }
557 return bestQuad == null ? null : _orderQuadCorners(bestQuad);
558 } finally {
559     contours.dispose();
560     hierarchy.dispose();
561 }
562 } finally {
563     dilated.dispose();
564 }
565 } finally {
566     kernel.dispose();
567 }
568 } finally {
569     edges.dispose();
570 }
571 } finally {
572     blurred.dispose();
573 }
574 }
575
576 /// Orders 4 arbitrary quad points as [topLeft, topRight, bottomLeft,
577 /// bottomRight]: top-left has the smallest x+y, bottom-right the largest;
578 /// top-right has the largest x-y, bottom-left the smallest.
579 List<(double, double)> _orderQuadCorners(List<(double, double)> pts) {
580     final bySum = [...pts]..sort((a, b) => (a.$1 + a.$2).compareTo(b.$1 + b.$2));
581     final byDiff = [...pts]..sort((a, b) => (a.$1 - a.$2).compareTo(b.$1 - b.$2));
582     return [bySum.first, byDiff.last, byDiff.first, bySum.last];
583 }
584
585 /// Bilinearly interpolates a fractional page position within the
586 /// quadrilateral [quad] ([topLeft, topRight, bottomLeft, bottomRight]).
587 (double, double) _bilinearInterpolate(List<(double, double)> quad, double xFrac, double yFrac) {
588     final tl = quad[0], tr = quad[1], bl = quad[2], br = quad[3];
589     final topX = tl.$1 + (tr.$1 - tl.$1) * xFrac;
590     final topY = tl.$2 + (tr.$2 - tl.$2) * xFrac;
591     final bottomX = bl.$1 + (br.$1 - bl.$1) * xFrac;
592     final bottomY = bl.$2 + (br.$2 - bl.$2) * xFrac;
593     return (topX + (bottomX - topX) * yFrac, topY + (bottomY - topY) * yFrac);
594 }
595
596 /// Searches the window around ([expectedX], [expectedY]) for the fiducial
597 /// mark. Rather than taking the largest dark blob in the window (which
598 /// reliably locked onto nearby header text/table lines instead of the
599 /// actual mark whenever those happened to out-area it - a real bug, not
600 /// just noise, since it picked the *same* wrong feature just as
601 /// confidently at higher photo resolution), this rejects implausibly
602 /// tiny (speckle) or large (a line/text run spanning much of the window)
603 /// blobs, then takes whichever remaining blob's centroid is closest to
604 /// the expected position.
605 cv.Point2f? _findMarkerNear(
606     cv.Mat gray,
607     double expectedX,
608     double expectedY, {
609     double windowFrac = _cornerSearchWindowFrac,
610 }) {
611     final windowRadius = windowFrac * math.min(gray.width, gray.height);
612     final left = (expectedX - windowRadius).clamp(0, gray.width - 1).round();
613     final top = (expectedY - windowRadius).clamp(0, gray.height - 1).round();
614     final right = (expectedX + windowRadius).clamp(left + 1, gray.width).round();
615     final bottom = (expectedY + windowRadius).clamp(top + 1, gray.height).round();
616
617     final roiRect = cv.Rect(left, top, right - left, bottom - top);
618     final roi = gray.region(roiRect);
619     try {
620         final roiMean = roi.mean();
621         double backgroundMean;
622         try {
623             backgroundMean = roiMean.val1;
624         } finally {
625             roiMean.dispose();
626         }
627
628         final clahe = cv.createCLAHE(clipLimit: 3, tileGridSize: (4, 4));
629         try {
630             final enhanced = clahe.apply(roi);
631             try {
632                 final blurred = cv.gaussianBlur(enhanced, (3, 3), 0);

```

```

632     try {
633         final roiMinDim = math.min(roi.width, roi.height);
634         final blockSize = ((roiMinDim ~/ 4) | 1).clamp(11, 51);
635         final binary = cv.adaptiveThreshold(
636             blurred,
637             255,
638             cv.ADAPTIVE_THRESH_GAUSSIAN_C,
639             cv.THRESH_BINARY_INV,
640             blockSize,
641             8,
642         );
643         try {
644             // Ordinary photo blur/JPEG softening can still break a
645             // printed mark's edges into fragments too small to
646             // individually clear the area filter below. A small dilation
647             // bridges those fragments back into one solid blob before
648             // contour extraction, with little effect on marks that were
649             // already solid.
650             final dilateKernel = cv.getStructuringElement(cv.MORPH_RECT, (3, 3));
651             try {
652                 final dilated = cv.dilate(binary, dilateKernel, iterations: 1);
653                 try {
654                     final (contours, hierarchy) = cv.findContours(dilated, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE);
655                     try {
656                         final roiArea = (right - left) * (bottom - top);
657                         final windowCenterX = (right - left) / 2;
658                         final windowCenterY = (bottom - top) / 2;
659
660                         double bestScore = double.infinity;
661                         cv.Rect? bestRect;
662                         for (final contour in contours) {
663                             final area = cv.contourArea(contour);
664                             if (area < 8 || area > roiArea * 0.35) continue;
665                             final rect = cv.boundingRect(contour);
666
667                             // Reject blobs that don't actually look like a printed
668                             // mark (a clean filled square or thin tick)  this is
669                             // the main defense against reporting "found" on
670                             // whatever dark clutter happens to sit near the
671                             // expected position when the camera isn't even
672                             // pointed at a sheet.
673                             final boxArea = (rect.width * rect.height).toDouble();
674                             if (boxArea <= 0 || boxArea < _markerMinFillRatio) continue;
675                             final longSide = math.max(rect.width, rect.height);
676                             final shortSide = math.max(1, math.min(rect.width, rect.height));
677                             if (longSide / shortSide > _markerMaxAspect) continue;
678
679                             final cx = rect.x + rect.width / 2;
680                             final cy = rect.y + rect.height / 2;
681                             final distance = math.sqrt(math.pow(cx - windowCenterX, 2) + math.pow(cy - windowCenterY,
682 2));
683
684                             final blobRoi = roi.region(rect);
685                             double contrast;
686                             try {
687                                 final blobMean = blobRoi.mean();
688                                 try {
689                                     contrast = backgroundMean - blobMean.val1;
690                                 } finally {
691                                     blobMean.dispose();
692                                 }
693                             } finally {
694                                 blobRoi.dispose();
695                             }
696                             if (contrast < _markerMinContrast) continue;
697
698                             // Prefer blobs near the expected center that are also dark
699                             // enough to be real fiducials, not shadow edges.
700                             final score = distance / math.max(contrast, _markerMinContrast);
701                             if (score < bestScore) {
702                                 bestScore = score;
703                                 bestRect = rect;
704                             }
705                         }
706                         if (bestRect == null) return null;
707                         final cx = left + bestRect.x + bestRect.width / 2;
708                         final cy = top + bestRect.y + bestRect.height / 2;
709                         return cv.Point2f(cx.toDouble(), cy.toDouble());
710                     } finally {
711                         contours.dispose();
712                         hierarchy.dispose();
713                     }
714                 } finally {
715                     dilated.dispose();
716                 }
717             } finally {
718                 dilateKernel.dispose();
719             }
720         } finally {
721             binary.dispose();
722         }

```

```
722         } finally {
723             blurred.dispose();
724         }
725     } finally {
726         enhanced.dispose();
727     }
728     } finally {
729         clahe.dispose();
730     }
731 } finally {
732     roi.dispose();
733 }
734 }
735
736 OmrScanResult _readBubbles(cv.Mat inkMap, OmrExamTemplate template, int canonicalWidth, int canonicalHeight) {
737     // Sample the full printed bubble (its radius, converted to canonical
738     // pixels), not a size guessed independently of what's actually on the
739     // page  otherwise enlarging bubbles in the sheet generator without a
740     // matching decoder change just keeps sampling the same small patch in
741     // the middle, which is where the printed choice letter lives.
742     final bubbleSampleHalfPx = template.bubbleRadiusPt * _canonicalPxPerPt;
743     final items = <OmrItemResult>[];
744     for (final section in template.sections) {
745         for (final itemNumber in section.items.keys.toList()..sort()) {
746             final choices = section.items[itemNumber]!;
747             String? bestChoice;
748             double bestFill = -1;
749             double runnerUpFill = -1;
750             for (final bubble in choices) {
751                 final fill = _bubbleFillFraction(inkMap, bubble, bubbleSampleHalfPx, canonicalWidth, canonicalHeight);
752                 if (fill > bestFill) {
753                     runnerUpFill = bestFill;
754                     bestFill = fill;
755                     bestChoice = bubble.choice;
756                 } else if (fill > runnerUpFill) {
757                     runnerUpFill = fill;
758                 }
759             }
760
761             if (bestFill < _blankFillFloor) {
762                 items.add(OmrItemResult(sectionName: section.name, itemNumber: itemNumber, markedChoice: null));
763             } else if (bestFill - runnerUpFill < _ambiguousMargin) {
764                 items.add(
765                     OmrItemResult(
766                         sectionName: section.name,
767                         itemNumber: itemNumber,
768                         markedChoice: null,
769                         isAmbiguous: true,
770                     ),
771                 );
772             } else {
773                 items.add(
774                     OmrItemResult(sectionName: section.name, itemNumber: itemNumber, markedChoice: bestChoice),
775                 );
776             }
777         }
778     }
779     return OmrScanResult(examCode: template.examCode, items: items);
780 }
781
782 /// Fraction (0-1) of the bubble's ring-shaped sample ROI that is "ink" on
783 /// the binarized [inkMap] (255 = ink, after THRESH_BINARY_INV). Samples
784 /// the outer bubble area minus a central zone where the printed choice
785 /// letter lives, so blank sheets aren't misread as ambiguous marks.
786 double _bubbleFillFraction(cv.Mat inkMap, BubblePos bubble, double sampleHalfPx, int canonicalWidth, int
canonicalHeight) {
787     final cx = bubble.xFrac * canonicalWidth;
788     final cy = bubble.yFrac * canonicalHeight;
789     final outerFill = _squareFillFraction(inkMap, cx, cy, sampleHalfPx, canonicalWidth, canonicalHeight);
790     final innerHalfPx = sampleHalfPx * _bubbleInnerSampleFrac;
791     final innerFill = _squareFillFraction(inkMap, cx, cy, innerHalfPx, canonicalWidth, canonicalHeight);
792
793     final outerArea = sampleHalfPx * sampleHalfPx * 4;
794     final innerArea = innerHalfPx * innerHalfPx * 4;
795     final ringArea = outerArea - innerArea;
796     if (ringArea <= 0) return outerFill;
797
798     return ((outerFill * outerArea) - (innerFill * innerArea)) / ringArea;
799 }
800
801 double _squareFillFraction(cv.Mat inkMap, double cx, double cy, double sampleHalfPx, int canonicalWidth, int
canonicalHeight) {
802     final left = (cx - sampleHalfPx).clamp(0, canonicalWidth - 1).round();
803     final top = (cy - sampleHalfPx).clamp(0, canonicalHeight - 1).round();
804     final right = (cx + sampleHalfPx).clamp(left + 1, canonicalWidth).round();
805     final bottom = (cy + sampleHalfPx).clamp(top + 1, canonicalHeight).round();
806
807     final roi = inkMap.region(cv.Rect(left, top, right - left, bottom - top));
808     try {
809         final mean = roi.mean();
810         try {
```

```
811         return mean.val1 / 255.0;
812     } finally {
813         mean.dispose();
814     }
815     } finally {
816         roi.dispose();
817     }
818 }
819 }
```

```

1 import 'dart:math' as math;
2
3 import 'package:camera/camera.dart';
4 import 'package:flutter/foundation.dart';
5 import 'package:flutter/material.dart';
6
7 import '../core/constants/app_colors.dart';
8 import '../core/omr/omr_decoder.dart';
9 import '../core/omr/omr_templates.dart';
10 import '../core/routes/app_routes.dart';
11 import '../core/state/app_state.dart';
12
13 class _AlignmentCheckRequest {
14   final String imagePath;
15   final OmrExamTemplate template;
16   const _AlignmentCheckRequest(this.imagePath, this.template);
17 }
18
19 AlignmentCheck _checkAlignment(_AlignmentCheckRequest request) {
20   return const OmrDecoder().locateCorners(request.imagePath, request.template);
21 }
22
23 class _LiveCornersRequest {
24   final Uint8List lumaBytes;
25   final int width;
26   final int height;
27   final int bytesPerRow;
28   final OmrExamTemplate template;
29   const _LiveCornersRequest(this.lumaBytes, this.width, this.height, this.bytesPerRow, this.template);
30 }
31
32 List<bool> _checkLiveCorners(_LiveCornersRequest request) {
33   return const OmrDecoder().checkCornersFromLuma(
34     request.lumaBytes,
35     request.width,
36     request.height,
37     request.bytesPerRow,
38     request.template,
39   );
40 }
41
42 /// OMR Scanner Loop  mirrors SCREENS.EXAM_SCANNING.
43 /// Shows a live camera feed inside a capture viewfinder with a live
44 /// per-corner alignment overlay, and lets the user photograph each answer
45 /// sheet ("Scan Next" captures + advances) before compiling the batch
46 /// ("Compile Data").
47 class ExamScanningScreen extends StatefulWidget {
48   const ExamScanningScreen({super.key});
49
50   @override
51   State<ExamScanningScreen> createState() => _ExamScanningScreenState();
52 }
53
54 class _ExamScanningScreenState extends State<ExamScanningScreen> with WidgetsBindingObserver {
55   CameraController? _cameraController;
56   Future<void>? _initializeFuture;
57   String? _cameraError;
58   bool _isCapturing = false;
59
60   /// Live, per-corner feedback: null until the first frame check completes,
61   /// then whether each of the 4 corner marks (in
62   /// [OmrExamTemplate.cornerMarkers] order) was found in the most recently
63   /// checked frame. This now also gates capture (see [_readyToCapture]) 
64   /// it isn't purely advisory any more  but the post-capture check in
65   /// [_capture] remains the authoritative gate, since it runs against the
66   /// actual full-resolution captured photo rather than these lower-effort
67   /// preview frames, and a page can still fail it even after this live
68   /// signal read all 4 as found.
69   List<bool>? _liveCornersFound;
70   DateTime? _lastFrameCheckAt;
71   bool _frameCheckInFlight = false;
72
73   /// When all 4 corners most recently read as found, continuously  set the
74   /// moment they first do, cleared the instant any check comes back with
75   /// fewer than 4. [_readyToCapture] requires this to have held for
76   /// [_requiredStableDuration] before allowing a capture, so a single
77   /// flickery "4/4" frame (a brief glare, a shaky hand) can't by itself
78   /// trigger a scan.
79   DateTime? _all4FoundSince;
80
81   /// How long all 4 corners must read as continuously found before capture
82   /// is allowed. Live checks land roughly every 600ms
83   /// ([_onCameraFrame]'s throttle), so this requires at least one
84   /// corroborating re-check beyond the frame that first read 4/4, not just
85   /// that single frame.
86   static const Duration _requiredStableDuration = Duration(milliseconds: 500);
87
88   /// Hard gate on capture: every corner must currently read as found *and*
89   /// have done so continuously for [_requiredStableDuration]. The "Scan
90   /// Next" button is disabled whenever this is false (see [build]), and
91   /// [_capture] re-checks it defensively before ever calling

```

```

92  /// `takePicture()`  there should be no path to a capture attempt while
93  /// this is false, let alone to a scan result.
94  bool get _readyToCapture {
95    final since = _all4FoundSince;
96    return since != null && DateTime.now().difference(since) >= _requiredStableDuration;
97  }
98
99  /// Neutral until the first live check completes, green once all 4
100  /// anchors are currently found, red otherwise (some or none found).
101  Color get _scanWindowBorderColor {
102    final found = _liveCornersFound;
103    if (found == null) return AppColors.accentYellowGreen;
104    return found.every((f) => f) ? AppColors.primaryGreen : AppColors.warmRedOrange;
105  }
106
107  @override
108  void initState() {
109    super.initState();
110    WidgetsBinding.instance.addObserver(this);
111    _initializeFuture = _setUpCamera();
112  }
113
114  Future<void> _setUpCamera() async {
115    setState(() => _cameraError = null);
116    try {
117      final cameras = await availableCameras();
118      if (cameras.isEmpty) {
119        setState(() => _cameraError = 'No camera was found on this device.');
```

```

183     _initializeFuture = _setUpCamera();
184     setState(() {});
185   }
186 }
187
188 @override
189 void dispose() {
190   WidgetsBinding.instance.removeObserver(this);
191   final controller = _cameraController;
192   if (controller != null && controller.value.isStreamingImages) {
193     controller.stopImageStream();
194   }
195   controller?.dispose();
196   super.dispose();
197 }
198
199 /// Throttled live check: at most once every 600ms, and never overlapping
200 /// a check already in flight, so this stays cheap enough to run
201 /// continuously while framing. Feeds both the on-screen indicator and
202 /// [_readyToCapture]'s stabilization tracking.
203 void _onCameraFrame(CameraImage image) {
204   if (!mounted || !_frameCheckInFlight) return;
205   final now = DateTime.now();
206   if (_lastFrameCheckAt != null && now.difference(_lastFrameCheckAt!) < const Duration(milliseconds: 600)) {
207     return;
208   }
209   final template = omrTemplates[AppStateScope.of(context).activeExamCode];
210   if (template == null) return;
211
212   _lastFrameCheckAt = now;
213   _frameCheckInFlight = true;
214   final plane = image.planes.first;
215   compute(
216     _checkLiveCorners,
217     _LiveCornersRequest(plane.bytes, image.width, image.height, plane.bytesPerRow, template),
218   ).then((found) {
219     _frameCheckInFlight = false;
220     if (!mounted) return;
221     setState(() {
222       _liveCornersFound = found;
223       if (found.every((f) => f)) {
224         _all4FoundSince ??= now;
225       } else {
226         _all4FoundSince = null;
227       }
228     });
229   }).catchError((_) {
230     _frameCheckInFlight = false;
231   });
232 }
233
234 Future<void> _capture(AppState appState) async {
235   final controller = _cameraController;
236   if (controller == null || !controller.value.isInitialized || !_isCapturing) return;
237   // Defensive re-check: the "Scan Next" button is already disabled unless
238   // this holds (see build()), but re-checking here means there's no path
239   // to takePicture() while corners aren't confidently, stably found 
240   // not just a button-state assumption.
241   if (!_readyToCapture) return;
242
243   setState(() => _isCapturing = true);
244   try {
245     final file = await controller.takePicture();
246     final template = omrTemplates[appState.activeExamCode];
247     if (template != null) {
248       final check = await compute(_checkAlignment, _AlignmentCheckRequest(file.path, template));
249       if (!check.aligned) {
250         // The live check above is advisory-strength (a lower-effort
251         // preview frame); this one runs the real decoder's corner search
252         // against the actual captured photo and is authoritative. No
253         // bypass  a photo that fails this is always discarded, never
254         // added to the batch, so an unconfirmed page geometry can never
255         // reach perspective correction, bubble sampling, or scoring.
256         if (!mounted) return;
257         await _showMisalignedDialog(check.message);
258         return;
259       }
260     }
261     if (!mounted) return;
262     appState.addCapturedPage(file);
263   } on CameraException catch (e) {
264     if (!mounted) return;
265     ScaffoldMessenger.of(context).showSnackBar(
266       SnackBar(content: Text('Capture failed: ${e.description ?? e.code}')),
267     );
268   } catch (e) {
269     if (!mounted) return;
270     ScaffoldMessenger.of(context).showSnackBar(
271       SnackBar(content: Text('Capture failed: $e')),
272     );
273   } finally {

```

```

274     if (mounted) setState(() => _isCapturing = false);
275   }
276 }
277
278 /// Informational only  always ends with the photo discarded. There is
279 /// no "use anyway" option: a page whose 4 corners weren't confirmed on
280 /// the actual captured photo is always rejected, per the hard
281 /// all-4-corners requirement.
282 Future<void> _showMisalignedDialog(String? message) {
283   return showDialog<void>(
284     context: context,
285     barrierDismissible: false,
286     builder: (context) => AlertDialog(
287       title: const Text('Page not fully detected'),
288       content: Text(message ?? 'Page not fully detected. Please align the sheet and try again.'),
289       actions: [
290         TextButton(onPressed: () => Navigator.of(context).pop(), child: const Text('Try Again')),
291       ],
292     ),
293   );
294 }
295
296 Future<void> _compileData(AppState appState) async {
297   await appState.processCapturedPages();
298   if (!mounted) return;
299   if (appState.scanProcessingError != null) {
300     ScaffoldMessenger.of(context).showSnackBar(
301       SnackBar(content: Text('Could not process the scan: ${appState.scanProcessingError}')),
302     );
303     return;
304   }
305   Navigator.of(context).pushReplacementNamed(AppRoutes.examinationResults);
306 }
307
308 static const _defaultCornerFractions = [(0.05, 0.05), (0.95, 0.05), (0.05, 0.95), (0.95, 0.95)];
309
310 @override
311 Widget build(BuildContext context) {
312   final appState = AppStateScope.of(context);
313   final activeTemplate = omrTemplates[appState.activeExamCode];
314   final cornerFractions = activeTemplate == null
315     ? _defaultCornerFractions
316     : activeTemplate.cornerMarkers.map((c) => (c.xFrac, c.yFrac)).toList();
317
318   // The 4 corner marks aren't necessarily near the page's literal
319   // (0,0)-(1,1) edges (a narrow bubble grid leaves them well inside the
320   // page  see the comment on OmrDecoder.decode's dstCorners), so the
321   // guide box's real-world aspect ratio has to come from the marks'
322   // actual bounding box, not the full page's aspect ratio, or the guide
323   // rectangle drawn on screen won't match the marks' true proportions.
324   final pageWidthPt = activeTemplate?.pageWidthPt ?? 595.28;
325   final pageHeightPt = activeTemplate?.pageHeightPt ?? 841.89;
326   final markerXs = [for (final c in cornerFractions) c.$1];
327   final markerYs = [for (final c in cornerFractions) c.$2];
328   final markerAspectRatio =
329     ((markerXs.reduce(math.max) - markerXs.reduce(math.min)) * pageWidthPt) /
330     ((markerYs.reduce(math.max) - markerYs.reduce(math.min)) * pageHeightPt);
331
332   return Scaffold(
333     backgroundColor: Colors.black,
334     body: SafeArea(
335       child: ListenableBuilder(
336         listenable: appState,
337         builder: (context, _) {
338           return Column(
339             children: [
340               // Top bar
341               Padding(
342                 padding: const EdgeInsets.all(14),
343                 child: Row(
344                   mainAxisAlignment: MainAxisAlignment.spaceBetween,
345                   children: [
346                     InkWell(
347                       onTap: () => Navigator.of(context).pop(),
348                       child: Container(
349                         width: 32,
350                         height: 32,
351                         decoration: BoxDecoration(color: Colors.black.withOpacity(0.4), shape: BoxShape.circle),
352                         child: const Icon(Icons.close, color: Colors.white, size: 16),
353                       ),
354                     Text(
355                       'Sheet Document #${appState.currentScannedPage + 1}',
356                       style: const TextStyle(
357                         color: Colors.white,
358                         fontSize: 12,
359                         fontWeight: FontWeight.bold,
360                         fontFamily: 'monospace',
361                       ),
362                     ),
363                   ],
364                 ),
365               const SizedBox(width: 32),

```

```

365     ],
366   ),
367 ),
368
369 // Scanning viewfinder
370 Expanded(
371   child: Padding(
372     padding: const EdgeInsets.symmetric(horizontal: 24),
373     child: ClipRect(
374       borderRadius: BorderRadius.circular(24),
375       child: Stack(
376         children: [
377           Positioned.fill(child: _buildCameraLayer()),
378           Positioned.fill(
379             child: Container(
380               decoration: BoxDecoration(
381                 borderRadius: BorderRadius.circular(24),
382                 border: Border.all(color: _scanWindowBorderColor.withOpacity(0.85), width: 3),
383               ),
384             ),
385           ),
386           if (_cameraController?.value.isInitialized == true) ...[
387             Positioned.fill(
388               child: IgnorePointer(
389                 child: CustomPaint(
390                   painter: _PageGuidePainter(cornerFractions, markerAspectRatio,
391                     _liveCornersFound),
392                 ),
393             ),
394             Positioned(
395               top: 14,
396               left: 0,
397               right: 0,
398               child: Center(
399                 child: Container(
400                   padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
401                   decoration: BoxDecoration(
402                     color: Colors.black.withOpacity(0.6),
403                     borderRadius: BorderRadius.circular(6),
404                     border: Border.all(color: AppColors.accentYellowGreen.withOpacity(0.2)),
405                   ),
406                   child: Text(
407                     _liveCornersFound == null
408                       ? 'Position sheet within the frame'
409                       : _readyToCapture
410                         ? 'Ready to scan'
411                         : _liveCornersFound!.every((f) => f)
412                           ? 'Hold steady'
413                           : _liveCornersFound!.any((f) => f)
414                             ? '$_liveCornersFound!.where((f) => f).length}/4 corner marks
locked'
415                               : 'Align all 4 corners inside the frame',
416                     style: const TextStyle(color: Color(0xFF6EE7B7), fontSize: 10, fontFamily:
'monospace'),
417                   ),
418                 ),
419               ),
420             ),
421           ],
422         ],
423       ),
424     ),
425   ),
426 ),
427
428 // Bottom action panel
429 Container(
430   width: double.infinity,
431   padding: const EdgeInsets.fromLTRB(18, 24, 18, 18),
432   decoration: BoxDecoration(
433     gradient: LinearGradient(
434       begin: Alignment.bottomCenter,
435       end: Alignment.topCenter,
436       colors: [Colors.black, Colors.black.withOpacity(0.95), Colors.transparent],
437     ),
438   ),
439   child: Container(
440     padding: const EdgeInsets.all(16),
441     decoration: BoxDecoration(
442       color: const Color(0xFF0F172A).withOpacity(0.9),
443       borderRadius: BorderRadius.circular(16),
444       border: Border.all(color: AppColors.slate800),
445     ),
446     child: Column(
447       children: [
448         Text(
449           appState.capturedPages.isEmpty
450             ? 'ALIGN SHEET AND CAPTURE'

```

```
451           : '${appState.capturedPages.length} SHEET${appState.capturedPages.length == 1 ? '' :
'S'} CAPTURED',
452           style: const TextStyle(color: Color(0xFF34D399), fontSize: 11, fontWeight:
FontWeight.bold, letterSpacing: 0.5),
453         ),
454         const SizedBox(height: 12),
455         Row(
456           children: [
457             Expanded(
458               child: OutlinedButton(
459                 onPressed: _cameraController?.value.isInitialized == true && !_isCapturing &&
_readyToCapture
460                   ? () => _capture(appState)
461                   : null,
462                 style: OutlinedButton.styleFrom(
463                   backgroundColor: AppColors.slate800,
464                   foregroundColor: Colors.white,
465                   disabledBackgroundColor: AppColors.slate800.withOpacity(0.4),
466                   padding: const EdgeInsets.symmetric(vertical: 12),
467                   side: BorderSide.none,
468                   shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
469                 ),
470                 child: _isCapturing
471                   ? const SizedBox(
472                     width: 14,
473                     height: 14,
474                     child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
475                   )
476                   : const Text('Scan Next', style: TextStyle(fontSize: 12, fontWeight:
FontWeight.w700)),
477             ),
478           ),
479           const SizedBox(width: 10),
480           Expanded(
481             child: ElevatedButton(
482               onPressed: appState.capturedPages.isNotEmpty && !appState.isProcessingScans
483                 ? () => _compileData(appState)
484                 : null,
485               style: ElevatedButton.styleFrom(
486                 backgroundColor: AppColors.primaryGreen,
487                 disabledBackgroundColor: AppColors.primaryGreen.withOpacity(0.35),
488                 foregroundColor: Colors.white,
489                 padding: const EdgeInsets.symmetric(vertical: 12),
490                 shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
491               ),
492               child: appState.isProcessingScans
493                 ? const SizedBox(
494                   width: 14,
495                   height: 14,
496                   child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
497                 )
498                 : const Text('Compile Data', style: TextStyle(fontSize: 12, fontWeight:
FontWeight.w700)),
499             ),
500           ),
501         ],
502       ),
503     ),
504   ),
505 ),
506 ),
507 ],
508 );
509 },
510 ),
511 ),
512 );
513 }
514
515 Widget _buildCameraLayer() {
516   if (_cameraError != null) {
517     return Container(
518       color: Colors.black,
519       alignment: Alignment.center,
520       padding: const EdgeInsets.all(24),
521       child: Column(
522         mainAxisAlignment: MainAxisAlignment.min,
523         children: [
524           const Icon(Icons.videocam_off_outlined, color: Colors.white54, size: 32),
525           const SizedBox(height: 12),
526           Text(
527             _cameraError!,
528             textAlign: TextAlign.center,
529             style: const TextStyle(color: Colors.white70, fontSize: 12),
530           ),
531           const SizedBox(height: 16),
532           OutlinedButton(
533             onPressed: () => setState(() => _initializeFuture = _setUpCamera()),
534             style: OutlinedButton.styleFrom(foregroundColor: Colors.white, side: const BorderSide(color:
Colors.white54)),
```

```

535         child: const Text('Retry'),
536       ),
537     ],
538   ),
539   );
540 }
541
542 return FutureBuilder<void>(
543   future: _initializeFuture,
544   builder: (context, snapshot) {
545     final controller = _cameraController;
546     if (snapshot.connectionState == ConnectionState.done &&
547         controller == null &&
548         _cameraError == null) {
549       return Container(
550         color: Colors.black,
551         alignment: Alignment.center,
552         padding: const EdgeInsets.all(24),
553         child: Column(
554           mainAxisAlignment: MainAxisAlignment.min,
555           children: [
556             const Icon(Icons.videocam_off_outlined, color: Colors.white54, size: 32),
557             const SizedBox(height: 12),
558             const Text(
559               'Could not start the camera.',
560               textAlign: TextAlign.center,
561               style: TextStyle(color: Colors.white70, fontSize: 12),
562             ),
563             const SizedBox(height: 16),
564             OutlinedButton(
565               onPressed: () => setState(() => _initializeFuture = _setUpCamera()),
566               style: OutlinedButton.styleFrom(backgroundColor: Colors.white, side: const BorderSide(color:
Colors.white54)),
567               child: const Text('Retry'),
568             ),
569           ],
570         ),
571       );
572     }
573     if (controller == null || !controller.value.isInitialized) {
574       return const ColoredBox(
575         color: Colors.black,
576         child: Center(child: CircularProgressIndicator(color: AppColors.accentYellowGreen)),
577       );
578     }
579     return SizedBox.expand(
580       child: FittedBox(
581         fit: BoxFit.cover,
582         child: SizedBox(
583           width: controller.value.previewSize!.height,
584           height: controller.value.previewSize!.width,
585           child: CameraPreview(controller),
586         ),
587       ),
588     );
589   },
590 );
591 }
592 }
593
594 /// Draws a dimmed spotlight sized to match the real proportions of the
595 /// rectangle spanned by the sheet's 4 fiducial marks (not the full page 
596 /// see [markerAspectRatio]), a visible outline on that rectangle so it
597 /// reads as a target even against a busy background, and one L-shaped
598 /// bracket per corner  colored individually from [cornersFound]  so the
599 /// user can see exactly *which* corner still needs adjusting instead of
600 /// only an aggregate "X/4" count. The scan window's own overall border
601 /// color (see [_ExamScanningScreenState._scanWindowBorderColor]) still
602 /// gives an at-a-glance all-4 cue; this adds the specific, per-corner one.
603 class _PageGuidePainter extends CustomPainter {
604   const _PageGuidePainter(this.cornerFractions, this.markerAspectRatio, this.cornersFound);
605
606   /// The active exam template's corner-marker fractional (xFrac, yFrac)
607   /// positions, in [top-left, top-right, bottom-left, bottom-right] order.
608   final List<(double, double)> cornerFractions;
609
610   /// Real-world width/height of the rectangle spanned by the 4 marks
611   /// (computed by the caller from the marks' own bounding box in page
612   /// points, not the full page's aspect ratio).
613   final double markerAspectRatio;
614
615   /// Live per-corner detection state, same order as [cornerFractions].
616   /// Null until the first live check completes, in which case every
617   /// bracket draws neutral.
618   final List<bool>? cornersFound;
619
620   static const double _insetFraction = 0.06;
621   static const double _bracketArmLength = 26;
622   static const double _bracketStrokeWidth = 4;
623
624   @override

```

```

625 void paint(Canvas canvas, Size size) {
626   final maxWidth = size.width * (1 - _insetFraction * 2);
627   final maxHeight = size.height * (1 - _insetFraction * 2);
628   double guideWidth = maxWidth;
629   double guideHeight = guideWidth / markerAspectRatio;
630   if (guideHeight > maxHeight) {
631     guideHeight = maxHeight;
632     guideWidth = guideHeight * markerAspectRatio;
633   }
634   final guideRect = Rect.fromCenter(
635     center: size.center(Offset.zero),
636     width: guideWidth,
637     height: guideHeight,
638   );
639
640   final dimPath = Path.combine(
641     PathOperation.difference,
642     Path().addRect(Offset.zero & size),
643     Path().addRect(guideRect),
644   );
645   canvas.drawPath(dimPath, Paint().color = Colors.black.withOpacity(0.55));
646
647   // A visible outline on the target rectangle itself  dimming alone
648   // reads as ambiguous against a busy background, this is what makes
649   // "the sheet goes exactly here" legible on its own, before any frame
650   // has even been checked.
651   canvas.drawRect(
652     guideRect,
653     Paint()
654       ..color = Colors.white.withOpacity(0.9)
655       ..style = PaintingStyle.stroke
656       ..strokeWidth = 1.5,
657   );
658
659   // One bracket per corner, positioned by normalizing each marker's page
660   // fraction against the 4 markers' own bounding box  the same
661   // normalization that gives markerAspectRatio its value (see the
662   // caller in build())  so a bracket lands exactly on that marker's
663   // real position within guideRect for any exam's layout, not just a
664   // generic corner of the box.
665   final minX = cornerFractions.map((c) => c.$1).reduce(math.min);
666   final maxX = cornerFractions.map((c) => c.$1).reduce(math.max);
667   final minY = cornerFractions.map((c) => c.$2).reduce(math.min);
668   final maxY = cornerFractions.map((c) => c.$2).reduce(math.max);
669
670   for (var i = 0; i < cornerFractions.length; i++) {
671     final (xFrac, yFrac) = cornerFractions[i];
672     final localX = maxX > minX ? (xFrac - minX) / (maxX - minX) : 0.5;
673     final localY = maxY > minY ? (yFrac - minY) / (maxY - minY) : 0.5;
674     final point = guideRect.topLeft + Offset(localX * guideRect.width, localY * guideRect.height);
675     final found = cornersFound != null && i < cornersFound!.length ? cornersFound![i] : null;
676     final color = found == null
677       ? Colors.white.withOpacity(0.85)
678       : found
679         ? AppColors.primaryGreen
680         : AppColors.warmRedOrange;
681     _drawBracket(canvas, point, towardRight: localX < 0.5, towardBottom: localY < 0.5, color: color);
682   }
683 }
684
685 /// Draws one L-shaped bracket at [point], arms extending toward the
686 /// guide rectangle's interior  [towardRight]/[towardBottom] say which
687 /// direction that is for this particular corner (e.g. the top-left
688 /// corner's arms extend right and down).
689 void _drawBracket(Canvas canvas, Offset point, {required bool towardRight, required bool towardBottom, required
Color color}) {
690   final dx = towardRight ? _bracketArmLength : -_bracketArmLength;
691   final dy = towardBottom ? _bracketArmLength : -_bracketArmLength;
692   final paint = Paint()
693     ..color = color
694     ..strokeWidth = _bracketStrokeWidth
695     ..strokeCap = StrokeCap.round
696     ..style = PaintingStyle.stroke;
697   canvas.drawLine(point, point + Offset(dx, 0), paint);
698   canvas.drawLine(point, point + Offset(0, dy), paint);
699 }
700
701 @override
702 bool shouldRepaint(covariant _PageGuidePainter oldDelegate) =>
703   !listEquals(cornerFractions, oldDelegate.cornerFractions) ||
704   markerAspectRatio != oldDelegate.markerAspectRatio ||
705   !listEquals(cornersFound, oldDelegate.cornersFound);
706 }

```

```

1 import 'dart:async';
2 import 'dart:io';
3
4 import 'package:camera/camera.dart';
5 import 'package:flutter/foundation.dart';
6 import 'package:flutter/material.dart';
7 import 'package:path_provider/path_provider.dart';
8 import '../models/activity_model.dart';
9 import '../models/answer_key.dart';
10 import '../models/cloud_file_model.dart';
11 import '../models/omr_scan_result.dart';
12 import '../omr/omr_decoder.dart';
13 import '../omr/omr_templates.dart';
14 import '../services/local_storage_service.dart';
15
16 class _OmrDecodeRequest {
17   final String imagePath;
18   final OmrExamTemplate template;
19   const _OmrDecodeRequest(this.imagePath, this.template);
20 }
21
22 OmrScanResult _decodeOmrPage(_OmrDecodeRequest request) {
23   return const OmrDecoder().decode(request.imagePath, request.template);
24 }
25
26 class _DebugVizRequest {
27   final String imagePath;
28   final OmrExamTemplate template;
29   final String outputDir;
30   final int pageIndex;
31   const _DebugVizRequest(this.imagePath, this.template, this.outputDir, this.pageIndex);
32 }
33
34 void _saveDebugVisualization(_DebugVizRequest request) {
35   const OmrDecoder().saveDebugVisualization(request.imagePath, request.template, request.outputDir,
36   request.pageIndex);
37 }
38
39 /// A lightweight in-memory app state shared across screens via
40 /// ChangeNotifierProvider-style access (kept dependency-free by
41 /// using InheritedNotifier through [AppStateScope]).
42 ///
43 /// This mirrors the mock `state` object used in the HTML prototype
44 /// (recentActivities, activeExamCode, sync timestamps, etc.).
45 class AppState extends ChangeNotifier {
46   String userRole = 'staff'; // 'staff' | 'admin'
47   String activeExamCode = 'AT'; // AT | PT | TAT | QTM
48
49   /// Manually-entered answer keys (see AnswerKeyEntryScreen), one per exam
50   /// code that's had a key saved.
51   final Map<String, AnswerKey> answerKeys = {};
52
53   String get answerKeyStatus => answerKeys.containsKey(activeExamCode) ? 'Loaded Success' : 'Not Uploaded';
54
55   String databaseLastSynced = 'June 15, 2026, 04:30 PM';
56   String localLastUpdated = 'June 16, 2026, 09:15 AM';
57
58   final LocalStorageService _localStorage = LocalStorageService();
59
60   /// The diagnostic batches/results registry shown on Staff Home. Persisted
61   /// to device storage via [_localStorage]  [loadPersistedData] populates
62   /// this at startup, and every mutation ([addBatch], [markBatchDone])
63   /// saves it back, so batches survive app restarts and version upgrades
64   /// rather than resetting to mock data every launch.
65   final List<ActivityModel> recentActivities = [];
66
67   /// Loads persisted app data. Called once at startup, before the first
68   /// frame, so the registry never flashes empty then populates.
69   Future<void> loadPersistedData() async {
70     final activities = await _localStorage.loadActivities();
71     recentActivities
72       ..clear()
73       ..addAll(activities);
74     final keys = await _localStorage.loadAnswerKeys();
75     answerKeys
76       ..clear()
77       ..addAll(keys);
78     notifyListeners();
79   }
80
81   final List<CloudFileModel> databaseCloudFiles = const [
82     CloudFileModel(name: 'Admission Exam - Batch 01-A (Synced)', code: 'AT', total: 50, timestamp: '06/04/2026'),
83     CloudFileModel(name: 'Personality Test - Batch 04 (Synced)', code: 'PT', total: 32, timestamp: '06/05/2026'),
84   ];
85
86   int currentScannedPage = 0;
87   int totalToScan = 50;
88
89   /// Raw captured sheet photos for the in-progress scan session, one per
90   /// page, in capture order. Cleared by [resetScanProgress].
91   final List<XFile> capturedPages = [];

```

```

91
92 /// Decoded bubble results for the in-progress scan session, one per
93 /// captured page, populated by [processCapturedPages]. Cleared by
94 /// [resetScanProgress].
95 final List<OmrScanResult> scannedResults = [];
96
97 bool isProcessingScans = false;
98 String? scanProcessingError;
99
100 /// Where the last processCapturedPages() run wrote debug visualization
101 /// images (corner-detection + bubble-grid overlays), if it managed to.
102 /// Diagnostic only  never blocks or affects real scan results.
103 String? lastDebugImagesDir;
104
105 List<ActivityModel> get pendingBatches =>
106     recentActivities.where((a) => a.status == 'Pending').toList();
107
108 void addBatch({required String title, required String typeCode}) {
109     String typeLabel = 'Admission Exam';
110     if (typeCode == 'PT') typeLabel = 'Personality Profile';
111     if (typeCode == 'TAT') typeLabel = 'Teaching Aptitude';
112     if (typeCode == 'QTM') typeLabel = 'Quantitative Math';
113
114     recentActivities.insert(
115         0,
116         ActivityModel(
117             type: typeLabel,
118             date: 'Pending Check',
119             batch: title,
120             status: 'Pending',
121             examCode: typeCode,
122         ),
123     );
124     notifyListeners();
125     _localStorage.saveActivities(recentActivities);
126 }
127
128 void markBatchDone(String batchName) {
129     final idx = recentActivities.indexWhere((a) => a.batch == batchName);
130     if (idx != -1) {
131         recentActivities[idx] = recentActivities[idx].copyWith(
132             status: 'Done',
133             date: 'Just now',
134         );
135         notifyListeners();
136         _localStorage.saveActivities(recentActivities);
137     }
138 }
139
140 void setActiveExamCode(String code) {
141     activeExamCode = code;
142     notifyListeners();
143 }
144
145 void setAnswerKey(AnswerKey key) {
146     answerKeys[key.examCode] = key;
147     unawaited(_localStorage.saveAnswerKeys(answerKeys));
148     notifyListeners();
149 }
150
151 void resetScanProgress() {
152     currentScannedPage = 0;
153     capturedPages.clear();
154     scannedResults.clear();
155     scanProcessingError = null;
156     notifyListeners();
157 }
158
159 /// Records a freshly captured sheet photo and advances the page counter.
160 void addCapturedPage(XFile file) {
161     capturedPages.add(file);
162     currentScannedPage = capturedPages.length;
163     notifyListeners();
164 }
165
166 /// Runs the OMR decoder over every page in [capturedPages] for the active
167 /// exam template, populating [scannedResults]. Each page is decoded on a
168 /// background isolate so the UI stays responsive.
169 Future<void> processCapturedPages() async {
170     final template = omrTemplates[activeExamCode];
171     if (template == null) {
172         scanProcessingError = 'No sheet layout is defined for exam code "$activeExamCode".';
173         notifyListeners();
174         return;
175     }
176
177     isProcessingScans = true;
178     scanProcessingError = null;
179     scannedResults.clear();
180     notifyListeners();
181

```

```
182     final debugDir = await _prepareDebugImagesDir();
183     lastDebugImagesDir = debugDir;
184
185     // Process every page independently: a failure on one sheet shouldn't
186     // hide debug output for it (debug images are most useful for exactly
187     // the pages that fail) or block decoding the rest of the batch.
188     final errors = <String>[];
189     var pageIndex = 0;
190     for (final page in capturedPages) {
191       pageIndex++;
192       try {
193         final result = await compute(_decodeOmrPage, _OmrDecodeRequest(page.path, template));
194         scannedResults.add(result);
195       } catch (e) {
196         errors.add('Sheet $pageIndex: $e');
197       }
198       if (debugDir != null) {
199         try {
200           await compute(_saveDebugVisualization, _DebugVizRequest(page.path, template, debugDir, pageIndex));
201         } catch (_) {
202           // Diagnostic-only; never let a debug-image failure block real results.
203         }
204       }
205     }
206
207     scanProcessingError = errors.isEmpty ? null : errors.join('\n');
208     isProcessingScans = false;
209     notifyListeners();
210   }
211
212   /// App-external "omr_debug" folder for [processCapturedPages]'s debug
213   /// visualization images. Returns null (silently) if unavailable rather
214   /// than failing the real scan over a diagnostic feature.
215   Future<String?> _prepareDebugImagesDir() async {
216     try {
217       final base = await getExternalStorageDirectory();
218       if (base == null) return null;
219       final dir = Directory('${base.path}/omr_debug');
220       if (!dir.existsSync()) dir.createSync(recursive: true);
221       return dir.path;
222     } catch (_) {
223       return null;
224     }
225   }
226
227   void syncLocalToDatabase() {
228     databaseLastSynced = localLastUpdated;
229     notifyListeners();
230   }
231 }
232
233 /// Simple InheritedNotifier-based scope so screens/widgets can access
234 /// [AppState] without pulling in a third-party state management package.
235 class AppStateScope extends InheritedNotifier<AppState> {
236   const AppStateScope({
237     super.key,
238     required AppState super.notifier,
239     required super.child,
240   });
241
242   static AppState of(BuildContext context) {
243     final scope = context.dependOnInheritedWidgetOfExactType<AppStateScope>();
244     assert(scope != null, 'AppStateScope not found in widget tree');
245     return scope!.notifier!;
246   }
247 }
```

```

1 // Generates the printable OMR answer sheets (answer_sheets/*.pdf) and the
2 // matching bubble-coordinate data (lib/core/omr/omr_templates.dart) from a
3 // single shared layout computation, so the two can never drift apart.
4 //
5 // Run with (from the repo root): cd tool && dart pub get && dart run generate_sheets.dart
6 //
7 // This is its own standalone Dart package (see tool/pubspec.yaml),
8 // deliberately kept out of the main app's pubspec.yaml dependency graph 
9 // pulling in the main app's opencv_dart/dartcv4 dependency here would make
10 // `dart run` try to build its native library for the desktop host (via
11 // CMake/Visual Studio), which this script has no need for and most dev
12 // machines won't have set up for that target.
13 //
14 // This replaces the original generate_sheets.py / gen_dart.py tooling,
15 // whose source (and the PDFs it produced) were never checked into this
16 // repo  only the Dart template data survived. This tool and its output
17 // are checked in this time.
18
19 import 'dart:io';
20 import 'dart:math' as math;
21
22 import 'package:pdf/pdf.dart';
23 import 'package:pdf/widgets.dart' as pw;
24
25 // -----
26 // Page geometry. Paper size is per-exam (see ExamSpec.pageWidthPt/
27 // pageHeightPt below)  AT prints on A4; QTM/TAT print on "long" bond
28 // paper, matching NDMU's actual forms. "Long" here is the Philippine long
29 // bond paper size (8.5 x 13in), not US Legal (8.5 x 14in)  a distinct,
30 // smaller size despite the same name.
31 // -----
32
33 const double kA4Width = 595.28;
34 const double kA4Height = 841.89;
35 const double kLongWidth = 612; // 8.5in
36 const double kLongHeight = 936; // 13in
37
38 /// Page edge -> outer edge of the corner markers. Printers generally can't
39 /// print all the way to the physical edge, so this stays as blank margin.
40 const double kPageMargin = 24;
41
42 /// Corner marker -> content bounding box. Kept small and uniform so the
43 /// scannable region hugs the actual content instead of the old design's
44 /// near-page-edge markers (up to ~470pt of dead space around narrow
45 /// sections like TAT/QTM).
46 const double kMarkerPad = 20;
47
48 /// Half-size of the drawn marker glyphs themselves (visual only, doesn't
49 /// affect layout math  the decoder locates markers by image analysis, not
50 /// by expecting an exact glyph size).
51 const double kMarkerHalf = 6;
52
53 const double kContentLeft = kPageMargin + kMarkerPad;
54 const double kContentTop = kPageMargin + kMarkerPad;
55
56 // -----
57 // Bubble grid geometry  sized close to the real official sheets' small,
58 // tight proportions (matching NDMU's actual QTM/TAT forms and the
59 // OLSAT/16PF-inspired AT/PT layouts), not enlarged for scan accuracy.
60 //
61 // An earlier version of this file enlarged bubbles relative to their
62 // printed letter, because the letter's own ink was making blank bubbles
63 // read as "maybe marked." That's no longer necessary: the decoder
64 // (_bubbleFillFraction in lib/core/omr/omr_decoder.dart) now samples an
65 // outer square minus an inner square around each bubble - isolating the
66 // ring where a pencil mark lands from the printed letter sitting in the
67 // center - independent of how small the bubble is. So sizing here can
68 // track the real sheets instead of working around a decoder limitation.
69 // -----
70
71 const double kChoicePitch = 15; // x spacing between adjacent choice bubbles
72 const double kRowPitch = 15; // y spacing between adjacent item rows
73 const double kColumnGap = 40; // extra x gap between one column and the next
74 const double kRowLabelWidth = 20; // space reserved for "12." row numbering
75
76 const double kBubbleRadius = 6;
77 const double kLetterFontSize = 6;
78 const int kRowsPerColumn = 30;
79
80 // -----
81 // Header block. Two kinds:
82 // - simple: GuideGrade's own generic brand header (AT/PT  these are only
83 //   structurally inspired by third-party commercial tests, so they keep
84 //   generic branding rather than reproducing OLSAT/16PF's own).
85 // - ndmu: the real NDMU Guidance Center letterhead + ID-field table +
86 //   scores block (QTM/TAT  NDMU's own documents, reproduced closely).
87 // Each exam reserves a fixed height for its own kind on every page, so its
88 // content bounding box  and therefore its corner markers  stays
89 // consistent regardless of how wide its bubble grid is.
90 // -----

```

```

91
92 enum HeaderKind { simple, ndmu }
93
94 const double kBrandRowHeight = 26;
95 const double kSubtitleRowHeight = 14;
96 const double kGapAfterSubtitle = 6;
97 const double kTableHeight = 34;
98 const double kGapAfterTable = 8;
99 const double kTitleRowHeight = 20;
100 const double kInstructionRowHeight = 12;
101 const double kGapBeforeGrid = 10;
102
103 const double kSimpleHeaderHeight =
104     kBrandRowHeight +
105     kSubtitleRowHeight +
106     kGapAfterSubtitle +
107     kTableHeight +
108     kGapAfterTable +
109     kTitleRowHeight +
110     kInstructionRowHeight +
111     kGapBeforeGrid;
112
113 // NDMU letterhead header: institutional heading box, 3-row ID table (Last
114 // Name/First Name/MI, School Last Attended, Address of School Last
115 // Attended), a Date/Birth/Age/Sex + Scores block, then the exam title.
116 const double kLetterheadHeight = 46;
117 const double kGapAfterLetterhead = 6;
118 const double kIdRowHeight = 22;
119 const int kIdRowCount = 3;
120 const double kGapAfterIdTable = 6;
121 const double kDateScoresHeight = 58;
122 const double kGapAfterDateScores = 8;
123 const double kNdmuTitleHeight = 28;
124 const double kNdmuInstructionHeight = 12;
125 const double kGapBeforeNdmuGrid = 8;
126
127 const double kNdmuHeaderHeight =
128     kLetterheadHeight +
129     kGapAfterLetterhead +
130     kIdRowHeight * kIdRowCount +
131     kGapAfterIdTable +
132     kDateScoresHeight +
133     kGapAfterDateScores +
134     kNdmuTitleHeight +
135     kNdmuInstructionHeight +
136     kGapBeforeNdmuGrid;
137
138 double headerHeightFor(HeaderKind kind) => switch (kind) {
139     HeaderKind.simple => kSimpleHeaderHeight,
140     HeaderKind.ndmu => kNdmuHeaderHeight,
141 };
142
143 /// Row 30's bubble center sits `(kRowsPerColumn - 1) * kRowPitch` below row
144 /// 1's, plus bubble radius clearance below that. Same for every exam  only
145 /// the header above it varies.
146
147 double contentHeightFor(ExamSpec exam) =>
148     headerHeightFor(exam.headerKind) + (exam.rowsPerColumn - 1) *
149     exam.rowPitch + 2 * exam.bubbleRadius;
150
151 // -----
152 // Brand colors, matching lib/core/constants/app_colors.dart.
153 // -----
154
155 final kGreen = PdfColor.fromHex('#2E7D32'); // AppColors.primaryGreen
156 final kRedOrange = PdfColor.fromHex('#E53935'); // AppColors.warmRedOrange
157 final kNavy = PdfColor.fromHex('#1A237E'); // AppColors.darkNavy
158 final kGray = PdfColor.fromHex('#757575'); // AppColors.textGray
159 final kBlack = PdfColors.black;
160
161 // -----
162 // Exam content  item counts and choice letters preserved from the current
163 // lib/core/omr/omr_templates.dart, except PT's "?" middle choice (almost
164 // certainly a lost-tooling artifact) is fixed to "b".
165 // -----
166
167 class SectionSpec {
168     final String name;
169     final int itemCount;
170     final List<String> Function(int itemNumberInSection) choicesForItem;
171     const SectionSpec(this.name, this.itemCount, this.choicesForItem);
172 }
173
174 class ExamSpec {
175     final String code;
176     final String title;
177     final List<SectionSpec> sections;
178     final HeaderKind headerKind;
179     final double pageWidthPt;
180     final double pageHeightPt;
181

```

```

182  /// Page geometry: how far the corner marks and bubble grid sit from the
183  /// page edges, and how the grid itself is spaced. Defaults match every
184  /// exam this tool has generated so far. A sheet with real, fixed
185  /// physical spacing that this tool didn't draw (an existing printed
186  /// form) overrides these to match its actual measurements instead.
187  final double pageMargin;
188  final double markerPad;
189  final double choicePitch;
190  final double rowPitch;
191  final double columnGap;
192  final double rowLabelWidth;
193  final double bubbleRadius;
194  final int rowsPerColumn;
195
196  /// Row count for just the first column, if it differs from every other
197  /// column  e.g. a samples/legend box eating part of the first column's
198  /// space, common on real answer sheets. Null (the default) means every
199  /// column, first included, uses [rowsPerColumn].
200  final int? firstColumnRows;
201
202  /// Direct override for [contentLeft]/[contentTop], for a sheet whose
203  /// measured content origin doesn't decompose cleanly into
204  /// pageMargin+markerPad (e.g. an existing printed form measured with a
205  /// ruler, where only the final distance from the page edge is known).
206  /// Null (the default) falls back to pageMargin + markerPad, as before.
207  final double? contentLeftOverride;
208  final double? contentTopOverride;
209
210  double get contentLeft => contentLeftOverride ?? (pageMargin + markerPad);
211  double get contentTop => contentTopOverride ?? (pageMargin + markerPad);
212
213  /// Direct override for where the 4 corner anchor marks are (top-left,
214  /// top-right, bottom-left, bottom-right  same order as everywhere
215  /// else), in points from the page's top-left. Null (the default) falls
216  /// back to the usual content-bounds-derived computation in
217  /// [_cornerMarkers]. Set this for a sheet whose anchors were physically
218  /// placed at known positions independent of content bounds (e.g. near
219  /// the sheet's actual physical corners, like a normal registration
220  /// mark, rather than hugging the bubble grid).
221  final List<(double, double)>? cornerMarkersOverride;
222
223  /// NDMU letterhead lines (only used when [headerKind] is
224  /// [HeaderKind.ndmu])  e.g. the center's name, university, and location.
225  final List<String> letterheadLines;
226
227  const ExamSpec(
228    this.code,
229    this.title,
230    this.sections, {
231      this.headerKind = HeaderKind.simple,
232      this.pageWidthPt = kA4Width,
233      this.pageHeightPt = kA4Height,
234      this.pageMargin = kPageMargin,
235      this.markerPad = kMarkerPad,
236      this.choicePitch = kChoicePitch,
237      this.rowPitch = kRowPitch,
238      this.columnGap = kColumnGap,
239      this.rowLabelWidth = kRowLabelWidth,
240      this.bubbleRadius = kBubbleRadius,
241      this.rowsPerColumn = kRowsPerColumn,
242      this.firstColumnRows,
243      this.contentLeftOverride,
244      this.contentTopOverride,
245      this.cornerMarkersOverride,
246      this.letterheadLines = const [],
247    });
248 }
249
250 final List<String> _atOdd = const ['A', 'B', 'C', 'D', 'E'];
251 final List<String> _atEven = const ['F', 'G', 'H', 'J', 'K'];
252 final List<String> _ptChoices = const ['a', 'b', 'c'];
253 final List<String> _abcd = const ['A', 'B', 'C', 'D'];
254 final List<String> _tf = const ['T', 'F'];
255
256 final List<ExamSpec> kExams = [
257   ExamSpec('PT', 'Personality Profile (PT)', [
258     SectionSpec('Personality Profile', 185, (n) => _ptChoices),
259   ]),
260   ExamSpec(
261     'TAT',
262     'Teaching Aptitude Test (TAT)',
263     [
264       SectionSpec('Test I', 30, (n) => _abcd),
265       SectionSpec('Test II', 80, (n) => _tf),
266       SectionSpec('Test III', 20, (n) => _tf),
267     ],
268     headerKind: HeaderKind.ndmu,
269     pageWidthPt: kLongWidth,
270     pageHeightPt: kLongHeight,
271     letterheadLines: const [
272       'Guidance, Honors, and Scholarship Center',

```

```

273     'Notre Dame of Marbel University',
274     'City of Koronadal, South Cotabato',
275   ],
276 ),
277 ExamSpec(
278   'QTM',
279   'Qualifying Test in Mathematics (QTM)',
280   [SectionSpec('Qualifying Test in Mathematics', 60, (n) => _abcd)],
281   headerKind: HeaderKind.ndmu,
282   pageWidthPt: kLongWidth,
283   pageHeightPt: kLongHeight,
284   letterheadLines: const [
285     'Guidance and Testing Center',
286     'NOTRE DAME OF MARBEL UNIVERSITY',
287     'City of Koronadal, South Cotabato',
288   ],
289 ),
290 // The Admission Test (AT)  a self-designed sheet, printed and laid out
291 // entirely by this tool on standard A4 (matches the paper it's actually
292 // printed on), the same way PT/QTM/TAT already are. An earlier version
293 // instead tried to match a real, physically-anchored OLSAT answer sheet
294 // NDMU had been administering  hand-measured with a ruler, then
295 // corrected by curve-fitting against a single reference photo, with a
296 // corner-marker position that was flagged provisional and never
297 // independently confirmed (see git history). Every one of those numbers
298 // was a potential source of systematic misalignment no amount of
299 // decoder tuning could fix, since the ground truth itself was uncertain.
300 // Printing our own sheet removes that uncertainty entirely: the PDF and
301 // the decoder's bubble coordinates below come from the exact same
302 // computation and can never disagree.
303 //
304 // Structure preserved from the original OLSAT-inspired design: 72
305 // items, choices alternating A-E / F-K by odd/even item.
306 //
307 // The default kBubbleRadius/kChoicePitch/kRowPitch (sized for QTM/TAT's
308 // dense multi-section grids) left this single 72-item section only 3
309 // columns of 30 rows on an A4 page  every page's column count is
310 // _maxColumnsFor's width-fitting max regardless of how many items
311 // actually need to fill it (see _layoutExam: itemsPerPage is always
312 // columnsPerPage * rowsPerColumn, and items fill columns sequentially,
313 // not spread evenly), so with rowsPerColumn=30 the 3rd column only
314 // needed 12 rows and the 4th went unused entirely  over half the page
315 // printed blank on the right and along the bottom. rowsPerColumn: 18
316 // uses exactly the 4 columns that fit this page width (72 / 4 = 18,
317 // divides evenly, no partially-empty column), and the larger
318 // bubbleRadius/choicePitch/rowPitch below scale the grid up to use most
319 // of the remaining page instead of leaving it blank  verified by
320 // rendering the regenerated PDF, not just computed on paper.
321 ExamSpec(
322   'AT',
323   'Admission Test (AT) - OLSAT',
324   [SectionSpec('Answer Document', 72, (n) => n.isOdd ? _atOdd : _atEven)],
325   bubbleRadius: 8,
326   choicePitch: 18,
327   rowPitch: 30,
328   rowsPerColumn: 18,
329 ),
330 ];
331
332 // -----
333 // Layout: shared by both the PDF drawing and the Dart template emission, so
334 // they can never disagree about where a bubble is.
335 // -----
336
337 /// One placed bubble, in top-left-origin point space (matches BubblePos'
338 /// fraction convention once divided by page width/height).
339 class BubblePoint {
340   final String choice;
341   final double x;
342   final double y;
343   const BubblePoint(this.choice, this.x, this.y);
344 }
345
346 /// One placed item (question), with its global item number within its
347 /// section and its bubbles.
348 class ItemPlacement {
349   final int itemNumber;
350   final List<BubblePoint> bubbles;
351   const ItemPlacement(this.itemNumber, this.bubbles);
352 }
353
354 /// One physical page: which section it continues, the 1-based page number
355 /// within that section (for multi-page sections), and its items.
356 class PagePlacement {
357   final SectionSpec section;
358   final int pageNumber;
359   final int pageCount;
360   final List<ItemPlacement> items;
361   const PagePlacement(this.section, this.pageNumber, this.pageCount, this.items);
362 }
363

```

```

364 class ExamLayout {
365   final ExamSpec exam;
366   final List<PagePlacement> pages;
367   final double contentWidth;
368   const ExamLayout(this.exam, this.pages, this.contentWidth);
369 }
370
371 double _columnWidth(int choiceCount, double rowLabelWidth, double choicePitch) =>
372   rowLabelWidth + (choiceCount - 1) * choicePitch;
373
374 /// Max columns that fit within the page's content-width budget for a given
375 /// per-item choice count.
376 int _maxColumnsFor(int choiceCount, ExamSpec exam) {
377   final budget = exam.pageWidthPt - 2 * exam.pageMargin - 2 * exam.markerPad;
378   final width = _columnWidth(choiceCount, exam.rowLabelWidth, exam.choicePitch);
379   return ((budget + exam.columnGap) / (width + exam.columnGap)).floor();
380 }
381
382 /// NDMU letterhead headers (QTM/TAT) need real horizontal room for their
383 /// institutional heading and ID fields regardless of how narrow that
384 /// exam's bubble grid is (QTM's 2-column A-D grid is barely 170pt wide) 
385 /// the real forms' letterhead spans most of the page width. Without this,
386 /// "NOTRE DAME OF MARBEL UNIVERSITY" and the Date/Scores block overflow
387 /// past a grid-width-only content box.
388 const double kNdmuMinContentWidth = 440;
389
390 ExamLayout _layoutExam(ExamSpec exam) {
391   final pages = <PagePlacement>[];
392   double widestContent = exam.headerKind == HeaderKind.ndmu ? kNdmuMinContentWidth : 0;
393   final firstColRows = exam.firstColumnRows ?? exam.rowsPerColumn;
394
395   // Maps a 0-based item index on a page to its (column, row), letting the
396   // first column be a different height than the rest (see
397   // ExamSpec.firstColumnRows). When firstColRows == rowsPerColumn, this
398   // reduces to the original indexOnPage ~/ rowsPerColumn / % rowsPerColumn
399   // math exactly.
400   //
401   // A short first column is bottom-aligned, not top-aligned: on OLSAT, a
402   // "SAMPLES" legend box occupies the *top* of column 1's space, so items
403   // 1-8 sit at the same height as items 17-24, 33-40 etc. (the *second*
404   // half of every other column), not aligned with 9-16 (the first half).
405   final firstColumnRowOffset = exam.rowsPerColumn - firstColRows;
406   (int, int) columnAndRow(int indexOnPage) {
407     if (indexOnPage < firstColRows) return (0, firstColumnRowOffset + indexOnPage);
408     final remaining = indexOnPage - firstColRows;
409     return (1 + remaining ~/ exam.rowsPerColumn, remaining % exam.rowsPerColumn);
410   }
411
412   for (final section in exam.sections) {
413     final choiceCount = section.choicesForItem(1).length;
414     final columnsPerPage = _maxColumnsFor(choiceCount, exam);
415     final itemsPerPage = firstColRows + (columnsPerPage - 1) * exam.rowsPerColumn;
416     final pageCount = (section.itemCount / itemsPerPage).ceil();
417
418     for (var pageIndex = 0; pageIndex < pageCount; pageIndex++) {
419       final firstItem = pageIndex * itemsPerPage + 1;
420       final lastItem = math.min(firstItem + itemsPerPage - 1, section.itemCount);
421       final (lastCol, _) = columnAndRow(lastItem - firstItem);
422       final columnsOnThisPage = lastCol + 1;
423       final usedWidth = columnsOnThisPage * _columnWidth(choiceCount, exam.rowLabelWidth, exam.choicePitch) +
424         (columnsOnThisPage - 1) * exam.columnGap;
425       if (usedWidth > widestContent) widestContent = usedWidth;
426
427       final items = <ItemPlacement>[];
428       for (var itemNumber = firstItem; itemNumber <= lastItem; itemNumber++) {
429         final indexOnPage = itemNumber - firstItem;
430         final (col, row) = columnAndRow(indexOnPage);
431         final columnX = exam.contentLeft +
432           col * (_columnWidth(choiceCount, exam.rowLabelWidth, exam.choicePitch) + exam.columnGap) +
433           exam.rowLabelWidth;
434         final rowY = exam.contentTop + headerHeightFor(exam.headerKind) + row * exam.rowPitch;
435         final choices = section.choicesForItem(itemNumber);
436         final bubbles = [
437           for (var i = 0; i < choices.length; i++) BubblePoint(choices[i], columnX + i * exam.choicePitch, rowY),
438         ];
439         items.add(ItemPlacement(itemNumber, bubbles));
440       }
441       pages.add(PagePlacement(section, pageIndex + 1, pageCount, items));
442     }
443   }
444
445   return ExamLayout(exam, pages, widestContent);
446 }
447
448 /// The 4 corner marker points (top-left-origin, points), sized to the
449 /// widest page this exam actually uses so every page's content  including
450 /// narrower sections in a multi-page exam like TAT  stays safely inside a
451 /// single template-level marker rectangle.
452 List<(double, double)> _cornerMarkers(ExamLayout layout) {
453   final exam = layout.exam;
454   if (exam.cornerMarkersOverride != null) return exam.cornerMarkersOverride!;

```

```

455     final right = exam.contentLeft + layout.contentWidth + exam.markerPad;
456     final bottom = exam.contentTop + contentHeightFor(exam) + exam.markerPad;
457     return [(exam.pageMargin, exam.pageMargin), (right, exam.pageMargin), (exam.pageMargin, bottom), (right,
bottom)];
458 }
459 // -----
460 // PDF drawing
461 // -----
462
463
464 void _paintPage(PdfGraphics canvas, ExamSpec exam, PagePlacement page, ExamLayout layout, PdfFont regular, PdfFont
bold) {
465     double flip(double topLeftY) => exam.pageHeightPt - topLeftY;
466
467     // Corner markers: solid filled squares at all 4 corners. An earlier
468     // version drew thin single-line ticks (2.5pt wide) at the top-right and
469     // bottom-right corners instead  visually distinct, but with roughly a
470     // fifth of the ink area of the solid squares, they were consistently the
471     // weakest link in corner detection (easily broken up by ordinary photo
472     // blur/JPEG softening). Same solid square everywhere removes that
473     // asymmetry rather than continuing to work around it in the decoder.
474     final corners = _cornerMarkers(layout);
475     canvas.setColor(kBlack);
476     for (final corner in corners) {
477         canvas.drawRect(corner.$1 - kMarkerHalf, flip(corner.$2) - kMarkerHalf, kMarkerHalf * 2, kMarkerHalf * 2);
478         canvas.fillPath();
479     }
480
481     switch (exam.headerKind) {
482         case HeaderKind.simple:
483             _paintSimpleHeader(canvas, exam, page, layout, regular, bold, flip);
484         case HeaderKind.ndmu:
485             _paintNdmuHeader(canvas, exam, page, layout, regular, bold, flip);
486     }
487
488     // Bubble grid. QTM/TAT (NDMU forms) draw slightly elongated ovals rather
489     // than perfect circles, matching those sheets' actual bubble style; the
490     // decoder samples square regions regardless of the drawn shape, so this
491     // is purely cosmetic.
492     final bubbleRx = exam.bubbleRadius;
493     final bubbleRy = exam.headerKind == HeaderKind.ndmu ? exam.bubbleRadius * 0.8 : exam.bubbleRadius;
494     for (final item in page.items) {
495         canvas.setColor(kBlack);
496         final labelX = item.bubbles.first.x - kRowLabelWidth;
497         canvas.drawString(regular, 9, '${item.itemNumber}.', labelX, flip(item.bubbles.first.y) - 3);
498         for (final bubble in item.bubbles) {
499             canvas.setLineWidth(1);
500             canvas.drawEllipse(bubble.x, flip(bubble.y), bubbleRx, bubbleRy);
501             canvas.strokePath();
502             canvas.setColor(kBlack);
503             final metrics = bold.stringMetrics(bubble.choice) * kLetterFontSize;
504             canvas.drawString(bold, kLetterFontSize, bubble.choice, bubble.x - metrics.advanceWidth / 2, flip(bubble.y) -
metrics.ascent / 2);
505         }
506     }
507 }
508
509 /// GuideGrade's own generic brand header (AT/PT)  these are only
510 /// structurally inspired by third-party commercial tests (OLSAT/16PF), so
511 /// they keep GuideGrade's own branding rather than reproducing those
512 /// tests' names or logos.
513 void _paintSimpleHeader(
514     PdfGraphics canvas,
515     ExamSpec exam,
516     PagePlacement page,
517     ExamLayout layout,
518     PdfFont regular,
519     PdfFont bold,
520     double Function(double) flip,
521 ) {
522     var y = kContentTop;
523     canvas.setColor(kRedOrange);
524     canvas.drawString(bold, 20, 'Guide', kContentLeft, flip(y + 20));
525     final guideWidth = (bold.stringMetrics('Guide') * 20).advanceWidth;
526     canvas.setColor(kGreen);
527     canvas.drawString(bold, 20, 'Grade', kContentLeft + guideWidth, flip(y + 20));
528     y += kBrandRowHeight;
529
530     canvas.setColor(kGray);
531     canvas.drawString(regular, 10, 'Guidance and Testing Center', kContentLeft, flip(y + 10));
532     y += kSubtitleRowHeight + kGapAfterSubtitle;
533
534     // Name / exam info table.
535     final tableWidth = layout.contentWidth;
536     final columns = [
537         ('Last Name', tableWidth * 0.24),
538         ('First Name', tableWidth * 0.22),
539         ('MI', tableWidth * 0.08),
540         ('Exam Code', tableWidth * 0.18),
541         ('Batch', tableWidth * 0.14),

```

```

542     ('Date', tableWidth * 0.14),
543 ];
544 canvas.setColor(kBlack);
545 canvas.setLineWidth(1);
546 canvas.drawRect(kContentLeft, flip(y + kTableHeight), tableWidth, kTableHeight);
547 canvas.strokePath();
548 var colX = kContentLeft;
549 for (final (label, w) in columns) {
550     canvas.drawLine(colX, flip(y), colX, flip(y + kTableHeight));
551     canvas.strokePath();
552     canvas.setColor(kGray);
553     canvas.drawString(regular, 7, label, colX + 3, flip(y + 10));
554     colX += w;
555 }
556 y += kTableHeight + kGapAfterTable;
557
558 // Title (+ page indicator for multi-page sections).
559 canvas.setColor(kNavy);
560 final title = page.pageCount > 1 ? '${exam.title} - ${page.section.name} (Page ${page.pageNumber} of
561 ${page.pageCount})' : '${exam.title} - ${page.section.name}';
562 canvas.drawString(bold, 13, title, kContentLeft, flip(y + 15));
563 y += kTitleRowHeight;
564
565 canvas.setColor(kGray);
566 canvas.drawString(regular, 8, 'Use a No. 2 pencil. Fill the circle completely.', kContentLeft, flip(y + 9));
567 }
568
569 /// NDMU's own official letterhead header (QTM/TAT)  institutional heading,
570 /// full ID-field table, and a Date/Birth/Age/Sex + Scores block, matching
571 /// the real forms these exam types are administered on today.
572 void _paintNdmuHeader(
573     PdfGraphics canvas,
574     ExamSpec exam,
575     PagePlacement page,
576     ExamLayout layout,
577     PdfFont regular,
578     PdfFont bold,
579     double Function(double) flip,
580 ) {
581     var y = kContentTop;
582     final width = layout.contentWidth;
583
584     // Letterhead box.
585     canvas.setColor(kBlack);
586     canvas.setLineWidth(1);
587     canvas.drawRect(kContentLeft, flip(y + kLetterheadHeight), width, kLetterheadHeight);
588     canvas.strokePath();
589     var ly = y + 13;
590     for (var i = 0; i < exam.letterheadLines.length; i++) {
591         final line = exam.letterheadLines[i];
592         final size = i == 1 ? 12.0 : 8.0; // the university name (line 2) stands out
593         final font = i == 1 ? bold : regular;
594         final metrics = font.stringMetrics(line) * size;
595         final textX = kContentLeft + (width - metrics.advanceWidth) / 2;
596         canvas.setColor(i == 1 ? kNavy : kGray);
597         canvas.drawString(font, size, line, textX, flip(ly));
598         ly += size + 4;
599     }
600     y += kLetterheadHeight + kGapAfterLetterhead;
601
602     // ID table: row 1 splits into Last Name / First Name / MI; rows 2-3 are
603     // full-width (School Last Attended, Address of School Last Attended).
604     canvas.setColor(kBlack);
605     canvas.setLineWidth(1);
606     final idTableHeight = kIdRowHeight * kIdRowCount;
607     canvas.drawRect(kContentLeft, flip(y + idTableHeight), width, idTableHeight);
608     canvas.strokePath();
609     for (var i = 1; i < kIdRowCount; i++) {
610         final lineY = y + kIdRowHeight * i;
611         canvas.drawLine(kContentLeft, flip(lineY), kContentLeft + width, flip(lineY));
612         canvas.strokePath();
613     }
614     final nameCols = [('Last Name', width * 0.45), ('First Name', width * 0.4), ('MI', width * 0.15)];
615     var colX = kContentLeft;
616     for (final (label, w) in nameCols) {
617         if (colX > kContentLeft) {
618             canvas.drawLine(colX, flip(y), colX, flip(y + kIdRowHeight));
619             canvas.strokePath();
620         }
621         canvas.setColor(kGray);
622         canvas.drawString(regular, 7, label, colX + 3, flip(y + 9));
623         colX += w;
624     }
625     canvas.setColor(kGray);
626     canvas.drawString(regular, 7, 'School Last Attended', kContentLeft + 3, flip(y + kIdRowHeight + 9));
627     canvas.drawString(regular, 7, 'Address of School Last Attended', kContentLeft + 3, flip(y + kIdRowHeight * 2 +
628 9));
629     y += idTableHeight + kGapAfterIdTable;
630
631     // Date/Birth/Age/Sex block (left) + exam-specific scores block (right).
632     final leftWidth = width * 0.55;

```

```

631 final rightX = kContentLeft + leftWidth + 10;
632 final rightWidth = width - leftWidth - 10;
633
634 canvas.setColor(kGray);
635 canvas.drawString(regular, 7, 'Date Today: Year ____ Month ____ Day ____', kContentLeft, flip(y + 10));
636 canvas.drawString(regular, 7, 'Birth Date: Year ____ Month ____ Day ____', kContentLeft, flip(y + 26));
637 canvas.drawString(regular, 7, 'Age: ____ Sex: M ( ) F ( )', kContentLeft, flip(y + 42));
638
639 canvas.setLineWidth(0.75);
640 if (exam.code == 'TAT') {
641   canvas.setColor(kBlack);
642   canvas.drawString(bold, 7, 'Raw', rightX + rightWidth * 0.45, flip(y + 8));
643   canvas.drawString(bold, 7, 'Scaled', rightX + rightWidth * 0.75, flip(y + 8));
644   const rows = ['Test I', 'Test II', 'Test III', 'Total'];
645   for (var i = 0; i < rows.length; i++) {
646     final rowY = y + 12 + i * 11.0;
647     canvas.setColor(kGray);
648     canvas.drawString(regular, 7, rows[i], rightX, flip(rowY + 8));
649     canvas.drawRect(rightX + rightWidth * 0.42, flip(rowY + 10), rightWidth * 0.22, 10);
650     canvas.strokePath();
651     canvas.drawRect(rightX + rightWidth * 0.72, flip(rowY + 10), rightWidth * 0.22, 10);
652     canvas.strokePath();
653   }
654 } else {
655   canvas.setColor(kNavy);
656   canvas.drawString(bold, 9, 'Scores', rightX, flip(y + 10));
657   canvas.setColor(kGray);
658   canvas.drawString(regular, 7, 'Raw Score', rightX, flip(y + 24));
659   canvas.drawRect(rightX, flip(y + 38), rightWidth * 0.42, 12);
660   canvas.strokePath();
661   canvas.drawString(regular, 7, 'Standard Score', rightX + rightWidth * 0.5, flip(y + 24));
662   canvas.drawRect(rightX + rightWidth * 0.5, flip(y + 38), rightWidth * 0.42, 12);
663   canvas.strokePath();
664   canvas.drawString(regular, 7, 'Test Booklet No.', rightX, flip(y + 54));
665   canvas.drawRect(rightX, flip(y + kDateScoresHeight), rightWidth * 0.42, 12);
666   canvas.strokePath();
667 }
668 y += kDateScoresHeight + kGapAfterDateScores;
669
670 // Title: exam name, then "<section>  ANSWER SHEET" (section omitted
671 // when it's just a restatement of the exam name, as for QTM's single
672 // section named the same as the exam itself).
673 final title1 = exam.title.replaceAll(RegExp(r'\s*\([A-Z]+\)'), '').toUpperCase();
674 final sectionDiffers = page.section.name.toUpperCase() != title1;
675 final title2Parts = [
676   if (sectionDiffers) page.section.name,
677   if (page.pageCount > 1) 'ANSWER SHEET (Page ${page.pageNumber} of ${page.pageCount})' else 'ANSWER SHEET',
678 ];
679 canvas.setColor(kNavy);
680 canvas.drawString(bold, 12, title1, kContentLeft, flip(y + 13));
681 canvas.drawString(bold, 9, title2Parts.join(' - ').toUpperCase(), kContentLeft, flip(y + 26));
682 y += kNdmuTitleHeight;
683
684 canvas.setColor(kGray);
685 canvas.drawString(regular, 7, 'Use a No. 2 pencil. Fill the circle completely.', kContentLeft, flip(y + 9));
686 }
687
688 // -----
689 // Dart template emission
690 // -----
691
692 const String _schema = '''
693 class BubblePos {
694   final String choice;
695   final double xFrac;
696   final double yFrac;
697   const BubblePos(this.choice, this.xFrac, this.yFrac);
698 }
699
700 class OmrSection {
701   final String name;
702   final int itemCount;
703   final Map<int, List<BubblePos>> items;
704   const OmrSection({required this.name, required this.itemCount, required this.items});
705 }
706
707 class OmrCorner {
708   final double xFrac;
709   final double yFrac;
710   const OmrCorner(this.xFrac, this.yFrac);
711 }
712
713 class OmrExamTemplate {
714   final String examCode;
715   final double pagewidthPt;
716   final double pageHeightPt;
717   /// Radius, in page points, of the printed bubble circles  the decoder
718   /// samples a region this size around each BubblePos, so it must match
719   /// the actual printed geometry rather than being guessed independently.
720   final double bubbleRadiusPt;
721   final List<OmrCorner> cornerMarkers;

```

```

722   final List<OmrSection> sections;
723   const OmrExamTemplate({
724     required this.examCode,
725     required this.pageWidthPt,
726     required this.pageHeightPt,
727     required this.bubbleRadiusPt,
728     required this.cornerMarkers,
729     required this.sections,
730   });
731 }
732 '';
733
734 String _formatFrac(double v) => v.toStringAsFixed(5);
735
736 String _emitTemplate(ExamLayout layout) {
737   final exam = layout.exam;
738   final varName = '_omr${exam.code}';
739   final corners = _cornerMarkers(layout);
740   final cornersDart = corners.map((c) => 'OmrCorner(${_formatFrac(c.$1 / exam.pageWidthPt)}, ${_formatFrac(c.$2 /
exam.pageHeightPt)})').join(', ');
741
742   // Merge all pages belonging to the same section back into one
743   // OmrSection (its items map spans every page it was laid out across).
744   final buffer = StringBuffer();
745   buffer.writeln('final OmrExamTemplate $varName = OmrExamTemplate(');
746   buffer.writeln('  examCode: "${exam.code}",');
747   buffer.writeln('  pageWidthPt: ${exam.pageWidthPt},');
748   buffer.writeln('  pageHeightPt: ${exam.pageHeightPt},');
749   buffer.writeln('  bubbleRadiusPt: ${exam.bubbleRadius},');
750   buffer.writeln('  cornerMarkers: const [$cornersDart],');
751   buffer.writeln('  sections: const [');
752   for (final section in exam.sections) {
753     buffer.writeln('    OmrSection(');
754     buffer.writeln('      name: "${section.name}",');
755     buffer.writeln('      itemCount: ${section.itemCount},');
756     buffer.writeln('      items: {');
757     final pagesForSection = layout.pages.where((p) => p.section == section);
758     for (final page in pagesForSection) {
759       for (final item in page.items) {
760         final bubblesDart = item.bubbles
761           .map((b) => 'BubblePos("${b.choice}", ${_formatFrac(b.x / exam.pageWidthPt)}, ${_formatFrac(b.y /
exam.pageHeightPt)})')
762           .join(', ');
763         buffer.writeln('        ${item.itemNumber}: [$bubblesDart],');
764       }
765     }
766     buffer.writeln('    },');
767     buffer.writeln('  ),');
768   }
769   buffer.writeln('  ],');
770   buffer.writeln('');
771   return buffer.toString();
772 }
773
774 // -----
775 // Entry point
776 // -----
777
778 Future<void> main() async {
779   // Paths are relative to this package's root (tool/), one level below the
780   // main app root  this script is a standalone Dart package (tool/pubspec.yaml)
781   // deliberately kept out of the main app's dependency graph.
782   final answerSheetsDir = Directory('..answer_sheets')..createSync(recursive: true);
783
784   final dartFile = StringBuffer();
785   dartFile.writeln('/// GENERATED by tool/generate_sheets.dart  do not hand-edit.');
```

```
810         ),
811       ),
812     );
813   }
814
815   final bytes = await pdf.save();
816   File('${answerSheetsDir.path}/${exam.code}.pdf').writeAsBytesSync(bytes);
817   stdout.writeln('Wrote answer_sheets/${exam.code}.pdf (${layout.pages.length} page(s), content width
${layout.contentWidth.toStringAsFixed(1)}pt)');
818
819   dartFile.writeln(_emitTemplate(layout));
820   mapEntries.add('${exam.code}': _omr${exam.code}');
821 }
822
823 dartFile.writeln('final Map<String, OmrExamTemplate> omrTemplates = {}');
824 for (final entry in mapEntries) {
825   dartFile.writeln('  $entry,');
826 }
827 dartFile.writeln(';');
828
829 File('../lib/core/omr/omr_templates.dart').writeAsStringSync(dartFile.toString());
830 stdout.writeln('Wrote lib/core/omr/omr_templates.dart');
831 }
```